

Free Linear Online Phase for Secure Multiparty Shuffle

Abstract. Shuffling is a fundamental operation in secure multiparty computation (MPC). However, the best-known maliciously secure n -party m -element shuffle protocols still incur at least n^2m online communication and computation, which exceeds the best-known $O(nm)$ result for the semi-honest additive secret sharing setting.

In this paper, we formalize the *permute-in-turn paradigm* for building MPC shuffle protocols from MPC permutation protocols, which captures the methodology implicitly adopted in most prior works. By further extending and enhancing this paradigm, we present generic transformations from semi-honest (resp. malicious) MPC permutation protocols to semi-honest (resp. malicious) MPC shuffle protocols that shuffle m items among n parties using only $O(n^2 + nm)$ online computation and communication¹. Our transformation is *for free* in two senses: it preserves the overall asymptotic complexity of the natural approach of performing n MPC permutations and yields an online phase with linear communication and computation complexity, while *freeing* the online phase of the shuffle protocol from depending on the implementation of the underlying MPC permutation protocol. This decoupling allows future work to optimize permutation protocols independently while benefiting all shuffle constructions following the paradigm.

Our results immediately yield the first maliciously secure shuffle with linear online communication and computation, under both additive secret sharing and Shamir secret sharing, improving upon the previous bounds of $O(Bn^2m)$ and $O(n^2m \log m)$, respectively². We formally prove semi-honest and universally composable (UC) security corresponding transformations. Our prototype demonstrates an over 20× improvement in online efficiency over prior state-of-the-art protocols when shuffling $m = 2^{12}$ items among $n = 15$ parties, with over 150× online communication savings and only a 1.1× slowdown in the offline phase. Given the widespread use of shuffling in MPC, our techniques are broadly applicable to practical protocols.

Keywords: multiparty computation · shuffle · random correlation

1 Introduction

Secure multiparty computation (MPC) has numerous real-world applications. In an MPC scheme, multiple parties jointly compute a function over their secret

¹ This is linear when the size m of data to be shuffled is asymptotically larger than the number of parties n , which we believe is common in practice.

² Prior to this work, there is no known (semi-honest) Shamir secret sharing shuffle protocol with $O(nm)$ online complexity.

inputs, while keeping each party’s input confidential from the others. This is particularly relevant in practical scenarios such as joint database queries [4, 10, 37], federated learning [24, 33, 36], and other collaborative data analytics.

A fundamental yet costly primitive in MPC is the shuffle operation, which allows the parties to jointly permute a secret-shared data array by a random permutation unknown to all parties. Shuffling serves as a core building block for more complex protocols, including MPC sorting [20, 21, 30], MPC oblivious RAM (ORAM) [28], and anonymous communication systems [3, 16]. Despite its importance, MPC shuffling remains a performance bottleneck in large-scale deployments [35].

While most existing works focus on optimizing the overall communication and computation of MPC shuffle protocols, when it comes to real-world efficiency, it is usually the online phase (i.e. data-dependent phase) that forms the bottleneck in MPC applications. However, despite these efforts, the online communication and computation of current maliciously secure MPC shuffle protocols remain over n^2m , for n parties shuffling m items [28, 31, 35]. As a concrete example, suppose we build an MPC-based anonymous communication system with the state-of-the-art maliciously secure MPC shuffle protocol of [35], which involves $n = 6$ servers shuffling batches of $m = 2^{12}$ client messages. According to our experiments (c.f. Section 7), the online phase alone would take over 25 seconds, which means the sent messages would experience a delay over 25 seconds. This large online latency represents the delay experienced by real-world users, which limits the practicality of MPC shuffle protocols in building anonymous communication systems.³ Also according to our experiments, after applying the techniques presented in this paper, the online phase would only take about 1 second, which is a significant improvement towards practical deployment.

We observe that nearly all existing MPC shuffle protocols share a common structure, which we call the *permute-in-turn paradigm*. In order to shuffle m items among n parties by a secret and uniformly random permutation π , each party P_i locally chooses a private random permutation π_i . After securely applying each permutation π_i to the data in sequence, the resulting permutation applied to the data is $\pi = \pi_n \circ \pi_{n-1} \circ \dots \circ \pi_1$, which is uniform and hidden from all parties.⁴ To securely apply each π_i to the data, the parties typically rely on an MPC permutation protocol Π_{perm} as a primitive. As will be discussed in more detail in Section 2, this simple idea and the primitive Π_{perm} underlie most existing MPC shuffle protocols [7, 9, 16, 22, 28, 29, 31, 34, 35].

³ Note that even though the offline computation requires roughly 400 seconds in our experiments, since these data-independent computations can be precomputed (and usually parallelized) before the messages of clients are known, they do not necessarily contribute to the delay experienced by the clients.

⁴ We note that π_i here, although conceptually private, is not an input for MPC shuffle protocols. A maliciously secure shuffle protocol must guarantee the secrecy of both $\mathbf{x}$ and π upon completion, but is allowed to leak π upon abort. This is different from MPC permutation protocols, where the permutation π_i is an input of P_i and must remain secret regardless of abort.

Although the paradigm is straightforward and already widely adopted, the n -party MPC shuffle protocols following this paradigm typically incur over n^2m communication and computation in their online (data-dependent) phase [7, 22, 28, 29, 31, 34, 35].⁵ This overhead stems from performing n times Π_{perm} in the online phase, each costing $O(nm)$. The only exceptions are the shuffle protocols by Eskandarian and Boneh [16] and Chen et al. [9], which avoid calling Π_{perm} in their online phases and achieve $O(nm)$ online complexity with a special type of correlated randomness called *shuffle correlation*. However, the constructions by [16] and [9] work only for semi-honest additive secret sharing.⁶ It thus remains an open problem to construct MPC shuffle protocols for malicious security with linear online communication and computation, and to generalize the semi-honest results by [9, 16] to other secret sharing schemes.

In this paper, we resolve this problem by presenting novel constructions that transform any semi-honest or maliciously secure MPC permutation protocol into an MPC shuffle protocol with matching security and with only $O(n^2 + nm)$ online communication and computation. Crucially, our constructions treat the permutation protocol as a black box, ensuring broad applicability across different MPC frameworks and secret sharing schemes. As suggested by the title, our transformation is *for free* in two senses: it preserves the overall asymptotic complexity⁷ necessary for performing n MPC permutations and yields linear online complexity, and it *frees* the online phase from depending on the specific implementation of Π_{perm} , enabling future work to optimize permutation protocols independently.

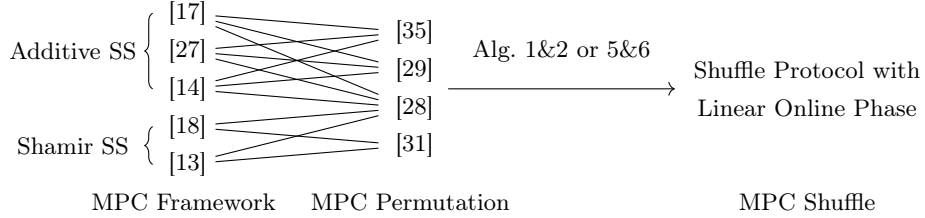


Fig. 1. Illustration of our transformation: any compatible pair of MPC framework and permutation protocol can be transformed into an MPC shuffle protocol with linear online complexity.

⁵ Note that [7, 22, 34] are two-party, where $n = 2$ is constant. Nevertheless, a straightforward generalization of these two-party protocols to the n -party case would also incur over n^2m online complexity, by performing $O(n^2)$ pairwise two-party shuffles, as is the case for [29], which generalizes [7] to n -party.

⁶ For readers interested in details about these constructions, as well as why extending them to malicious security is challenging, please see Section D for a brief review.

⁷ Our maliciously secure shuffle roughly doubles the concrete offline communication. Given that the offline phase can be precomputed, we believe this increase is moderate and well justified by the significant online efficiency gain.

Besides preserving asymptotic communication and computation complexity, our transformations also enjoy broad applicability. Figure 1 (non-exhaustively) illustrates how they work: by choosing a semi-honest (resp. malicious) MPC framework on the left and a compatible permutation protocol in the middle, and finally applying our corresponding semi-honest (resp. maliciously) secure transformation, one automatically obtains an MPC shuffle protocol on the right with linear online complexity, whose security level and offline complexity match the choice of the MPC framework and the permutation protocol. Given that the only drawback of our transformations is the doubled offline cost in the malicious setting, we believe our techniques can be widely adopted to improve the online efficiency and thus the practicality of MPC shuffle protocols.

Our contributions are summarized as follows.

1. We formalize the permute-in-turn paradigm and propose generic transformations from MPC permutation protocols to MPC shuffle protocols with linear online complexity for both semi-honest and malicious security. Since the permute-in-turn paradigm underlies most existing MPC shuffle protocols, our transformations are widely applicable.
2. Instantiating our constructions with state-of-the-art permutation protocols, we derive the first maliciously secure shuffles with $O(nm)$ online complexity for both additive and Shamir secret sharing. This improves prior bounds of $O(Bn^2m)$ [35] and $O(n^2m \log m)$ [28], respectively, where B is an additional parameter introduced by the cut-and-choose technique used in [35]. Note also there is no known semi-honest MPC shuffle protocol for Shamir secret sharing with $O(nm)$ online complexity prior to our work.
3. We prove semi-honest and universally composable (UC) security under the $\mathcal{F}_{\text{MPC}}$ -hybrid model, guaranteeing composability across MPC frameworks. Here $\mathcal{F}_{\text{MPC}}$ is an ideal functionality that provides black-box access to basic MPC operations and an MPC permutation protocol.
4. Experimental results confirm our theoretical analysis. Our prototype shows an over 20× speedup compared to state-of-the-art constructions, with only a moderate increase in offline overhead.

The rest of this paper is organized as follows. In Section 2 we offer a detailed discussion of related work, which also shows the broad existence of permute-in-turn paradigm. In Section 3, we introduce the primitives required for our constructions, formalize the permute-in-turn paradigm for building MPC shuffle protocols and present the definition of security. In Section 4, we describe our core insights for building shuffle protocols with linear online complexity. Section 5 shows our construction for semi-honest transformation, and Section 6 covers the malicious case. A sketch of the security proof is provided at the end of Section 6 to provide intuition regarding malicious security, and a rigorous proof can be found in Section C. In Section 7, we discuss our experimental results. Due to the page limit, we defer several important contents to the appendix. We present case studies of several candidates that can be used to instantiate our construction to obtain MPC shuffle protocols for different MPC frameworks in Section A. The formal proofs of semi-honest and malicious security are presented in Section B

and Section C, respectively. In Section D, we review existing shuffle correlation constructions in detail and compare them with ours. Finally, Section E discusses several practical issues regarding the use of the shuffle protocol.

2 Related Work

The main focus of this section is to provide an overview of existing work on MPC shuffle protocols, as well as to argue the broad existence of the *permute-in-turn paradigm*. To this end, we start by reviewing classical shuffle protocols, then move on to MPC-based shuffle protocols, and finally discuss the latest advances and ideas in developing MPC shuffle protocols.

Classical shuffles and mix-nets. The first shuffle protocol dates back to the seminal work of Chaum [8], which introduces the concept of mix-nets and implements anonymous communication in a server-aided scheme, assuming semi-honest servers. The work also inspires a series of follow-up studies on the concept of “decryption shuffle”, which involves a sender encrypting its messages with a sequence of public keys, each belonging to one of the servers. The servers then decrypt and permute the messages in sequence, and finally output the shuffled plaintext. The construction requires a linear communication overhead and a moderate amount of public-key encryptions and decryptions.

This result is later extended to malicious security by a number of works [1, 2, 19, 23]. The most commonly used primitive for the enhancement is zero-knowledge proofs, where each server proves that it has permuted the ciphertext honestly, while hiding the applied permutation. This approach, while being effective, is generally computationally intensive.

MPC-based shuffle protocols. In the context of MPC, Chase et al. [7] design a shuffle protocol for the semi-honest two-party setting, which inspires a series of follow-up efforts towards further optimizing MPC shuffle protocols. At the core of their construction is a protocol that generates correlated randomness between the two parties, requiring only $O(m \log m)$ communication for m items. This generation protocol is concretely efficient, as the only public-key primitive involved is the oblivious transfer extension (OTE).

While Chase’s protocol is two-party, it is later extended to the n -party setting. Laud [29] proposes an extension to an n -party shuffle protocol with $O(n^2m)$ online communication in the malicious security setting. Also building upon [7], Eskandarian and Boneh [16] and Chen et al. [9] both construct n -party shuffle protocols with only $O(nm)$ online complexity. Eskandarian and Boneh further enhance their construction, arguing that the improved version achieves malicious security. Thus, these works suggest that a maliciously secure MPC shuffle with linear online complexity is already within reach.

Malicious security and the quadratic barrier. However, a recent study by Song et al. [35] shows that the constructions proposed in [29] and [16] are in fact not

secure against malicious adversaries. Through a selective-abort attack, Song et al. show that a malicious adversary can bypass correctness checks of [29] and [16] with non-negligible probability and learn partial information about the applied permutation. This makes the shuffle non-uniform in the view of a malicious adversary, thereby breaking the security of the protocols.

Such an attack puts constructing maliciously secure MPC shuffle protocols in a dilemma: on one hand, the malicious adversary can tamper with the messages attempting to break security; on the other hand, it is not clear how such tampering of messages can be detected, let alone detecting it with linear online complexity.⁸ To at least address one side of the dilemma, Song et al. design a maliciously secure shuffle protocol for MPC with $O(Bn^2m)$ online communication complexity, where B is a parameter arising from the cut-and-choose technique. Thus, all currently known maliciously secure MPC shuffle protocols have at least quadratic n^2m online communication and computation complexities.

These results are summarized in Table 1.

Table 1. Existing MPC Shuffle Protocols

Protocol	Offline Complexity	Online Complexity	Security	Framework
[31]	$O(1)$	$O(2^n n^{1.5} m \log m)$	malicious	Threshold
[28]	$O(n^2 m \log m)$	$O(n^2 m \log m)$	malicious	Arbitrary
[29]	$O(n^2 m \log m)$	$O(n^2 m)$	semi-honest	SPDZ
[16]	$O(n^2 m \log m + n^3 m)$	$O(nm)$	semi-honest	SPDZ
[35]	$O(Bn^2 m \log m)$	$O(Bn^2 m)$	malicious	SPDZ
ours	$O(n^2 m \log m)$	$O(n^2 + nm)$	malicious	Arbitrary
ours	$O(Bn^2 m \log m)$	$O(n^2 + nm)$	malicious	SPDZ

The table lists upper-bounds for both communication and computation complexity. For all protocols listed, the online communication and computation complexity are identical.

n denotes the number of parties and m the number of items shuffled. B is the cut-and-choose parameter from [35], which grows with the security parameter.

The table assumes the offline complexity of multiplication is $O(n)$ in SPDZ [17].

Correlated randomness and recent advances. In more recent studies, the correlated randomness used in [7] has been identified and formalized as “correlated oblivious permutation” (COP) [22], or “permutation correlation” [34]. Building upon this concept, these works further improve the efficiency of generating such randomness. Han et al. [22] further identify redundancy in generating the correlation using random OT (ROT) and halve the communication cost. Peceny et

⁸ For completeness, we provide a brief discussion on the construction of [16] and the attack proposed by Song et al. in Section D.

al. [34] propose a new construction for generating correlated randomness with $O(n)$ communication, based on MPC-friendly PRF and learning parity with noise (LPN) assumption. These two constructions are built for semi-honest two-party computation, and it remains an open question whether they can be efficiently extended to n -party malicious security.

The permute-in-turn paradigm. We note that all the above constructions follow the same structure for building MPC shuffle protocols, which we call the *permute-in-turn paradigm*. That is, in each such construction, one can identify a component that is responsible for applying a permutation to the secret-shared data (i.e., a permutation protocol), where the permutation is chosen by one of the parties. The final construction thus consists of n such components, each corresponding to one party, executed in sequence. All the constructions mentioned so far, including early works [1, 2, 8, 19, 23] and more recent ones [7, 9, 16, 22, 29, 34, 35], follow this paradigm, even though many were not originally designed for multiparty computation. Although [31] adopts a slightly different approach, it can be viewed as a variant of the paradigm, where each group of parties permutes the data in turn (cf. Section A). However, to the best of our knowledge, all current maliciously secure shuffle protocols [28, 31, 35] execute the online phase of the underlying permutation protocol directly, where parties repeatedly form valid shares of the m permuted secrets n times among n parties, resulting in at least n^2m communication and computation. This makes the online phase both communication- and computation-heavy, which is undesirable for real-world applications that require quick responses.

Summary. In short, besides all following the permute-in-turn paradigm, existing MPC shuffle protocols either (i) achieve linear online complexity only with semi-honest security and additive secret sharing [9, 16], or (ii) support malicious security at the cost of quadratic online overhead [28, 31, 35]. This leaves open the central question we address in this paper: *how to design a semi-honest (resp. malicious) secure MPC shuffle protocol with linear online complexity and compatibility across secret-sharing schemes?*

3 Preliminary

3.1 Basic Notations

Throughout this paper, it is assumed that all elements and operations in multiparty computation protocols are in field $\mathbb{F}$, with $|\mathbb{F}| \geq 2^\kappa$ for any fixed statistical security parameter κ . Suppose there are n parties $P_1, P_2, \dots, P_n$, and m field elements to shuffle. Denote by

$$[t] := \{1, 2, \dots, t\},$$

the set of positive integers ranging from 1 to t for any positive integer t .

We use lowercase letters (e.g. x, y, z) for integers, and bold letters (e.g. $\mathbf{x}, \mathbf{y}, \mathbf{z}$) for vectors. We denote by $\mathbf{x}(i)$ or x_i the i -th entry of vector $\mathbf{x}$. The notation $\mathbf{x}(i)$ is useful when handling vectors with indices, e.g. vector $\mathbf{x}_1$ and $\mathbf{x}_2$. As we are mostly dealing with vectors of length m , we assume all vectors are of length m , unless otherwise specified. For an m -permutation $\pi : [m] \rightarrow [m]$, define

$$\pi(\mathbf{x}) := (x_{\pi(1)}, x_{\pi(2)}, \dots, x_{\pi(m)}).$$

Note that permuting is additively homomorphic, i.e.

$$\pi(\mathbf{x} + \mathbf{y}) = \pi(\mathbf{x}) + \pi(\mathbf{y}),$$

where the addition of two vectors is defined component-wise.

The composition of two permutations π_2, π_1 is another permutation, denoted as $\pi_2 \circ \pi_1$ and satisfies

$$\pi_2 \circ \pi_1(\mathbf{x}) = \pi_2(\pi_1(\mathbf{x})).$$

For convenience in later notation, for the composition of a sequence of permutations $\pi_1, \pi_2, \dots, \pi_n$, we denote

$$\bar{\pi}_i := \pi_i \circ \pi_{i-1} \circ \dots \circ \pi_1.$$

Executing a sub-protocol is written as:

$$\Pi(P_i : x, y, \llbracket z \rrbracket),$$

where Π is the name of the protocol; parameter “ $P_i : x$ ” means that this protocol takes a private input x from party P_i ; parameter y means that this protocol takes a public constant y from all parties; parameter $\llbracket z \rrbracket$ means that z is a value stored at $\mathcal{F}_{\text{MPC}}$. We will explain the meaning of “stored at $\mathcal{F}_{\text{MPC}}$ ” in the next subsection.

3.2 Primitives

We assume several secure MPC primitives, including inputting a secret, opening a secret, generating random values, computing additions and multiplications over $\mathbb{F}$. We also assume a secure MPC permutation protocol, which allows one of the parties to input a secret permutation and securely permute a vector. We also assume secure pair-wise channel and a public broadcast channel, which is a standard practice in the MPC literature [18].

To formalize, we follow the approach adopted by Escudero et al. [15], where an ideal arithmetic MPC functionality denoted by $\mathcal{F}_{\text{MPC}}$ is viewed as a Turing machine with internal state. The functionality interacts honestly with all parties, taking inputs and (restricted) commands from them and updating its internal state accordingly. We assume that the ideal arithmetic MPC functionality $\mathcal{F}_{\text{MPC}}$ supports the following commands:

- $\Pi_{\text{input}}(P_i : x, \text{id})$, which takes as input a field element x from P_i and stores it as (id, x) . “id” is a unique identifier that is agreed upon by all parties, which can be seen as a memory address of $\mathcal{F}_{\text{MPC}}$.

- $\Pi_{\text{input}}(y, \text{id})$, which takes as input a public constant field element y and stores it as (id, y) .
- $\Pi_{\text{rand}}(\text{id})$, which draws a uniform random value $r \in \mathbb{F}$ and stores it as (id, r) .
- $\Pi_{\text{add}}(\text{id}, \text{id}_1, C)$, which retrieves (id_1, x) from the memory and stores $(\text{id}, x + C)$, where C is a public constant.
- $\Pi_{\text{add}}(\text{id}, \text{id}_1, \text{id}_2)$, which retrieves (id_1, x_1) and (id_2, x_2) from the memory and stores $(\text{id}, x_1 + x_2)$.
- Π_{mul} works the same as Π_{add} , except it computes multiplications.
- $\Pi_{\text{open}}(\text{id})$, which retrieves (id, x) and outputs x to the adversary. If the adversary replies with “continue”, then $\mathcal{F}_{\text{MPC}}$ also sends x to honest parties; otherwise it sends “abort” to the honest parties.
- $\Pi_{\text{open}}(P_i, \text{id})$, which retrieves (id, x) and outputs x to party P_i .
- $\Pi_{\text{broadcast}}(P_i : x)$, which takes x from party P_i and broadcasts it to all parties. The communication complexity of broadcasting one field element is assumed to be $O(n)$. Note the assumption of secure pair-wise and broadcast channels is a standard practice in MPC literature [18].
- $\Pi_{\text{perm}}(P_i : \pi, (\text{id}_j)_{j=1}^m, (\text{id}'_j)_{j=1}^m)$, which takes a secret m -permutation from P_i , retrieves (id_j, x_j) and stores $(\text{id}'_j, x_{\pi(j)})$.
- $\Pi_{\text{abort}}()$, which sends “abort” to all parties. This command is invoked by the adversary.

In practice, under the semi-honest setting, we do not need Π_{mul} . $\mathcal{F}_{\text{MPC}}$ can thus be instantiated with Shamir secret sharing or additive secret sharing without MAC. For the malicious setting, $\mathcal{F}_{\text{MPC}}$ may be instantiated with additive secret sharing (e.g. [14, 26, 27]), Shamir’s secret sharing (e.g. [5, 13]), replicated secret sharing [11, 32], or other MPC frameworks, based on specific security requirements and application scenarios.

Note that we assume that $\mathcal{F}_{\text{MPC}}$ checks the correctness of the arguments. For example, if Π_{perm} is invoked with a non-permutation, $\mathcal{F}_{\text{MPC}}$ will output “abort” to all parties. During the execution, the malicious adversary can also invoke Π_{abort} at any time to abort the protocol.

For notational simplicity, we denote by $\llbracket x \rrbracket$ a secret value x stored by $\mathcal{F}_{\text{MPC}}$. This can also be understood as “sharing x among all parties”, since $\mathcal{F}_{\text{MPC}}$ is implemented by secure multiparty computation in practice. To distinguish between semi-honest and malicious settings, we write $\langle x \rangle$ for secret stored at maliciously secure ideal functionality. We will use $\llbracket \cdot \rrbracket$ only in semi-honest setting and $\langle \cdot \rangle$ only in malicious setting. We also denote

$$\llbracket d \rrbracket \leftarrow \llbracket a \rrbracket \cdot \llbracket b \rrbracket + \llbracket c \rrbracket$$

the process of computing $a \cdot b + c$ and store it in ideal functionality. Note that each symbol corresponds to a distinct identifier for $\mathcal{F}_{\text{MPC}}$, which allows us to omit the identifier in later description. For vector $\mathbf{x} = (x_1, x_2, \dots, x_m)$, we denote $\llbracket \mathbf{x} \rrbracket$ as storing each entry separately in ideal MPC. Hence, we also denote

$$\llbracket \mathbf{z} \rrbracket \leftarrow \llbracket a \rrbracket \cdot \llbracket \mathbf{x} \rrbracket + \llbracket \mathbf{y} \rrbracket$$

the process of computing $ax_1 + y_1, \dots, ax_m + y_m$ and storing them as $\mathbf{z}$.

Similarly, we write

$$\llbracket \mathbf{x}' \rrbracket \leftarrow \Pi_{\text{perm}}(P_i : \pi, \llbracket \mathbf{x} \rrbracket),$$

where

$$\mathbf{x}' = \pi(\mathbf{x}) = (x_{\pi(1)}, x_{\pi(2)}, \dots, x_{\pi(m)}).$$

For malicious security, we assume also a version of batched permutation protocol, where

$$\langle \pi(\mathbf{x}_1), \dots, \pi(\mathbf{x}_t) \rangle \leftarrow \Pi_{\text{perm}}(P_i : \pi, \langle \mathbf{x}_1, \dots, \mathbf{x}_t \rangle),$$

where $\langle \mathbf{x}_1, \dots, \mathbf{x}_t \rangle$ is an abbreviation for $(\langle \mathbf{x}_1 \rangle, \dots, \langle \mathbf{x}_t \rangle)$. This is a natural requirement for a permutation protocol, since in real-world application, what is to be shuffled is usually a list of vectors (e.g. rows of the database) rather than a list of field elements. As concrete examples, all protocols in [7, 16, 29, 31, 35] support such an operation.

3.3 Permute-in-Turn Paradigm

To shuffle a secret-shared vector $\llbracket \mathbf{x} \rrbracket$ into some $\llbracket \pi(\mathbf{x}) \rrbracket$ with secret, uniformly random π , most previous works implicitly follow a permute-in-turn paradigm and build MPC shuffle protocols from MPC permutation protocols.

To illustrate, suppose we have built a protocol *Permute* that securely implements Π_{perm} , i.e.

$$\llbracket \pi(\mathbf{x}) \rrbracket \leftarrow \text{Permute}(P_i : \pi, \llbracket \mathbf{x} \rrbracket).$$

Then the shuffle protocol can be implemented via sequential invocations of *Permute*, where each party P_i selects a random permutation π_i , and parties compute sequentially for $i = 1, 2, \dots, n$

$$\llbracket \mathbf{y}_i \rrbracket \leftarrow \text{Permute}(P_i : \pi_i, \llbracket \mathbf{y}_{i-1} \rrbracket),$$

where $\mathbf{y}_0 := \mathbf{x}$. Note that

$$\llbracket \mathbf{y}_n \rrbracket = \llbracket \pi(\mathbf{x}) \rrbracket = \llbracket \pi_n \circ \pi_{n-1} \circ \dots \circ \pi_1(\mathbf{x}) \rrbracket,$$

where π is not known to any party. In addition, as long as at least one honest party P_i has chosen its permutation π_i uniformly random, the resulting π will be uniformly random. Hence, due to the adversary lacking access to the π_i chosen by honest parties, the resulting π is independent of the adversary's view, which accomplishes the security goal of the shuffle protocol.

We formalize such an approach as the *permute-in-turn paradigm*: to build a shuffle protocol, we first implement a permutation protocol, and then use it to construct the shuffle protocol. Former works mostly concentrated on the design of the permutation protocol [7, 28, 29, 31, 35], while the paradigm itself is scarcely discussed or questioned.

Although the above construction works effectively, its online complexities are at least n times of that of `Permute`, since there are n sequential invocations. This problem becomes more severe when the online overhead of `Permute` is itself super-linear. As concrete examples, the implementation in [28] incurs $O(nm \log m)$ online overhead and [35] incurs $O(Bnm)$, which is then translated to $O(n^2m \log m)$ and $O(n^2Bm)$ online overhead in shuffle protocol, respectively.

What is firstly achieved in this paper is rewriting the above naive permute-in-turn paradigm by two phases, an offline phase `Shuffleoff` and an online phase `Shuffleon`, and moving all invocations to Π_{perm} to the offline phase. The online complexities of our constructions, besides now becoming $O(n^2 + nm)$, are also therefore independent of the implementation of Π_{perm} , while most constructions in previous works are not.

3.4 Security Model

Throughout this paper, we consider a static adversary, i.e. the corrupted parties are chosen and fixed before protocol starts. We consider the case of dishonest majority, where an adversary can corrupt up to $n - 1$ parties.¹

Semi-honest security. Our first construction considers semi-honest security, under $\mathcal{F}_{\text{MPC}}$ -hybrid model where $\mathcal{F}_{\text{MPC}}$ is semi-honest secure. In the semi-honest security, the adversary corrupting some parties is assumed to follow the protocol honestly, but nevertheless may do some extra computations to gain information about other party's input.

Definition 1 (Semi-honest Secure Shuffle). *Suppose there is a shuffle protocol `Shuffle`, which takes as input a vector $\llbracket \mathbf{x} \rrbracket$ stored at $\mathcal{F}_{\text{MPC}}$, and π_i from each party P_i , where each party P_i chooses π_i independently and uniformly. Parties follow the protocol honestly, and finally makes $\mathcal{F}_{\text{MPC}}$ store*

$$\llbracket \pi(\mathbf{x}) \rrbracket = \llbracket \pi_n \circ \pi_{n-1} \circ \cdots \circ \pi_1(\mathbf{x}) \rrbracket.$$

Suppose adversary $\mathcal{A}$ corrupts the parties in set $T \subsetneq [n]$, and will follow the protocol honestly. If for any $\mathbf{x}$ and $\{\pi_i\}_{i \in T}$,

$$I(\mathbf{x}, \text{view}_{\text{Shuffle}}^T(\mathbf{x}, \pi_1, \pi_2, \dots, \pi_n); (\pi_i)_{i \notin T}) = 0,$$

then the protocol `Shuffle` is said to be semi-honest secure, where I is the mutual information, $\text{view}_{\text{Shuffle}}^T$ is the view of corrupted parties in one execution of protocol `Shuffle` and π_i is the input of party P_i .

In other words, $(\pi_i)_{i \notin T}$ is independent (in a probability theory sense) of $\mathbf{x}$ and the view of adversary.

¹ Note that assuming dishonest majority here is without loss of generality, as our security proof does not rely on the specific number t of corrupted parties, besides the trivial requirement that $0 \leq t < n$. If we want to instantiate $\mathcal{F}_{\text{MPC}}$ with Shamir secret sharing under honest majority, setting $t = \lfloor (n-1)/2 \rfloor$ will do. In practice, the only concern is that the implementation of $\mathcal{F}_{\text{MPC}}$ must match the threat model.

In above definition, π_i is the input of party P_i , which is by design a uniformly random permutation and is the only input of party P_i in our construction. In later security proof, we will prove semi-honest security for protocol

$$\text{Shuffle} = \text{Shuffle}_{\text{on}} \circ \text{Shuffle}_{\text{off}},$$

i.e. the composition of two phases of shuffle protocol is secure.

Note that a standard simulation-based definition can also be given for semi-honest security. We choose the above information-theoretic definition for its simplicity and directness.

Malicious security. Our second construction considers security against a malicious adversary, who now can arbitrarily deviate from the protocol specification. Below is a sketch of the universally composable (UC) security definition for maliciously secure shuffle protocol. For more formal treatment and definitions, please refer to Appendix C.

Definition 2 (Universally Composable Security [6], Sketch). *Let $\mathcal{E}$ be an environment and $\mathcal{A}$ be an adversary that controlling the corrupted parties $P_i \in T \subseteq [n]$. Let ideal functionality $\mathcal{F}$ be $\mathcal{F}_{\text{MPC}}$ equipped with an additional ideal command Π_{Shuffle} . This command takes as input a vector $\mathbf{x}$, then samples π uniformly and stores $\pi(\mathbf{x})$ at $\mathcal{F}$. Let protocol Shuffle be an implementation of the command Π_{Shuffle} .*

Consider the following two games. One happens between $\mathcal{E}$, $\mathcal{A}$, $\mathcal{F}_{\text{MPC}}$ and honest parties $P_i \in \bar{T} = \{P_i \notin T\}$, executing real protocol Shuffle . Another happens between $\mathcal{E}$, simulator $\mathcal{S}$ and ideal functionality $\mathcal{F}$, simulating the view of adversary in the ideal execution. In each game, $\mathcal{E}$ chooses inputs of all honest parties, and sends them to either $P_i \in \bar{T}$ or $\mathcal{F}$. When the corrupted parties controlled by $\mathcal{A}$ need to send a message, $\mathcal{E}$ decides it, and when $\mathcal{A}$ receives anything, it reports to $\mathcal{E}$. In this real-world execution, if the protocol does not abort, $\mathcal{E}$ receives outputs of honest parties from honest parties.

In the ideal world, $\mathcal{S}$ will not receive the input of any party. Nevertheless, $\mathcal{S}$ needs to deduce the purported inputs of the corrupted parties, send them to $\mathcal{F}$, and is then informed by $\mathcal{F}$ of the output of the protocol for corrupted parties. If $\mathcal{E}$ does not demand $\mathcal{S}$ to abort, and the protocol ends without abort, $\mathcal{S}$ sends “continue” to $\mathcal{F}$, who then sends all outputs of honest parties to $\mathcal{E}$.

$\mathcal{E}$ continues interacting with $\mathcal{A}/\mathcal{S}$ during the entire process, while gaining information and doing its own computation. When $\mathcal{E}$ halts, it outputs a bit, representing its guess on which game it is playing.

The protocol Shuffle securely implements Π_{Shuffle} , if there exists a simulator $\mathcal{S}$, such that $\mathcal{E}$ cannot distinguish what game it is playing, i.e.

$$|\Pr[1 \leftarrow (\mathcal{E} \sqsubseteq \Pi_{\bar{T}, \mathcal{A}})] - \Pr[1 \leftarrow (\mathcal{E} \sqsubseteq \mathcal{F}_{\bar{T}, \mathcal{S}})]| < \epsilon = O(2^{-\kappa}).$$

Intuitively, the above definition guarantees that under $\mathcal{F}_{\text{MPC}}$ -hybrid model, the vector $\mathbf{x}$ are hidden from the adversary, whether the protocol aborts or not. It's worth noting that the π_i sampled by honest party P_i (in permute-in-turn

paradigm) is treated as internal randomness of the protocol: it is protected if the protocol does not abort, but can be leaked if the protocol aborts. This is in accordance with the security definition, since ideal command Π_{shuffle} only takes $\mathbf{x}$ as input.²

We remark that, the above definition considers only a dummy adversary, which acts according to $\mathcal{E}$'s command and sends everything it receives to $\mathcal{E}$. This is equivalent to a more "intelligent adversary", since $\mathcal{E}$ could perform all computations and decide the (malicious) actions. Also, the definition does not limit the computation resource of the environment, and hence achieves statistical security. Note that since we are working under $\mathcal{F}_{\text{MPC}}$ -hybrid model, this level of security is achievable with dishonest majority. We will present a more formal definition before presenting a formal security proof in the appendix, Section C.

4 Technical Overview

4.1 High-Level Idea

Figure 2 below illustrates the idea behind our shuffle protocols, assuming there are four parties.

Our basic idea is similar to the decryption shuffle by Chaum [8], where each server randomly permutes and decrypts with its own private key in turn. In our protocol, the parties pass around a masked vector $\mathbf{y}_0 = \mathbf{x} + \mathbf{z}_0$, where each party P_i applies its own secret permutation π_i and encrypts it with a one-time pad $\mathbf{z}_i$ to hide the permutation. P_i then sends

$$\mathbf{y}_i \leftarrow \pi_i(\mathbf{y}_{i-1} + \mathbf{z}_i) \quad (4.1)$$

to the next party. At the end, P_n broadcasts its $\mathbf{y}_n$ and all parties jointly decrypt the message by removing the accumulated mask Δ and share the shuffled vector $\mathbf{y} = \pi_n \circ \dots \circ \pi_1(\mathbf{x})$, where

$$\llbracket \Delta \rrbracket = \llbracket \pi_n(\pi_{n-1}(\dots \pi_1(\mathbf{z}_0 + \mathbf{z}_1) \dots + \mathbf{z}_{n-1}) + \mathbf{z}_n) \rrbracket \quad (4.2)$$

is computed in the offline phase.³

To achieve malicious security, we require each party P_i to send an additional message $\hat{\mathbf{y}}_i$ along with $\mathbf{y}_i$, implementing a hidden relation between $\mathbf{y}_i$ and $\hat{\mathbf{y}}_i$. By design, we require that

$$\beta \mathbf{y}_i = \hat{\mathbf{y}}_i + \mathbf{s}_i, \quad (4.3)$$

² In fact, under $\mathcal{F}_{\text{MPC}}$ -hybrid model, we can guarantee the security of π_i even if the protocol aborts, as we prove in Section C. This slightly weaker definition that allows leakage of π_i upon abort is due to a gap between the ideal functionality $\mathcal{F}_{\text{MPC}}$ and its real-world implementations, which we will discuss in Section E.1.

³ In MPC literature, such pre-shared random values are commonly referred to as *correlated randomness* or *correlation*. We will formalize the requirements over $\mathbf{z}_i$ and Δ in Definition 3, where we define semi-honest shuffle correlation.

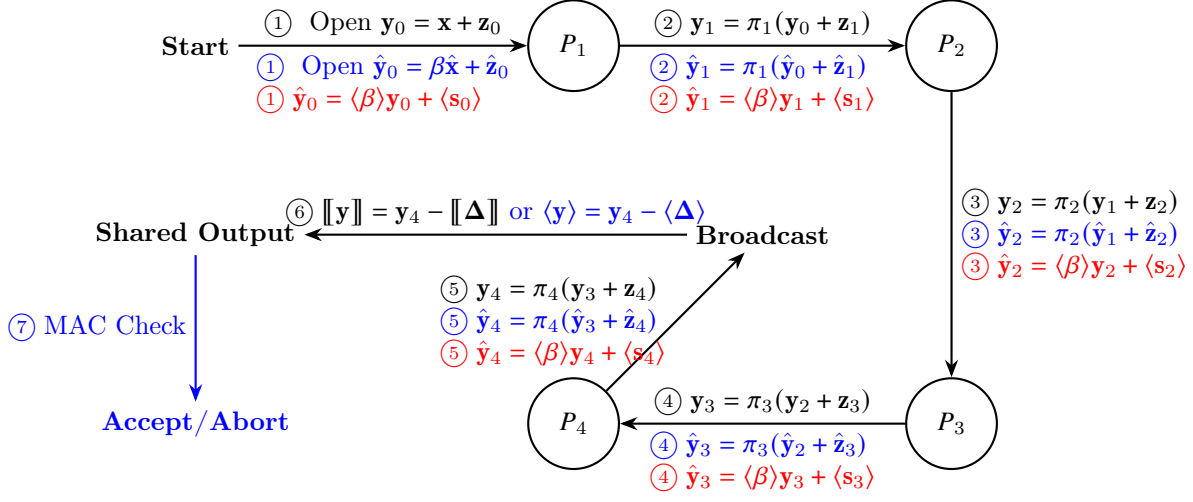


Fig. 2. Overview of online phases of our shuffle protocols for 4 parties. **Black texts** indicate messages and steps performed for both semi-honest and malicious security. **Blue texts** indicate additional messages and steps for malicious security. **Red texts** indicate the hidden correlation between y_i and $\hat{y}_i$, i.e. what the protocol expects $\hat{y}_i$ to be, if all parties are honest. It's worth noting that in the malicious setting, the final $\langle y \rangle$ is only securely shared without being opened, before the MAC check.

for a secret $\langle \beta \rangle$ and a secret random vector $\langle s_i \rangle$. At the end, after sharing y_n , each P_i also checks if the y_{i-1} and $\hat{y}_{i-1}$ it receives satisfy Equation 4.3.

Intuitively, since β and s_i are hidden behind the curtain of $\mathcal{F}_{\text{MPC}}$ from all parties, no malicious party can learn them and forge y_i and $\hat{y}_i$ that satisfy Equation 4.3, while all parties can jointly check Equation 4.3 with access to $\mathcal{F}_{\text{MPC}}$.

However, many challenges remain even given this blueprint. A by-definition approach to compute Δ is to invoke a secure permutation protocol Π_{perm} n times sequentially, which results in $O(n)$ offline round complexity. Also, a by-definition approach to implement the MAC checks is to let P_i share y_{i-1} and all parties run MPC multiplications over all m entries, which results in $O(n^2m)$ communication and computation. Moreover, even if these two efficiency issues are fixed, the security against malicious adversaries remains non-trivial. In particular, it is unclear why Equation 4.3 is sufficient to defend against malicious parties who can deviate arbitrarily from the protocol.

Hence, there are still three remaining challenges⁴:

1. How to generate the shuffle correlation, in particular Δ , efficiently with Π_{perm} ?

⁴ We omit here for brevity the problem of implementing the hidden relation. See Definition 4 for formal definition and requirements over these variables.

2. How to check the hidden relation (Equation 4.3) efficiently with low communication and computation?
3. Why does checking Equation 4.3 alone guarantee security against malicious adversaries?

4.2 Generating Shuffle Correlation Efficiently

To generate Δ efficiently, we decouple the permutations chosen by parties by decoupling the randomness used at each step. For semi-honest security, we let

$$\llbracket \mathbf{r}_i \rrbracket \leftarrow \Pi_{\text{rand}}(), \text{ for } i = 0, \dots, n, \quad (4.4)$$

$$\llbracket \mathbf{z}_0 \rrbracket \leftarrow \llbracket \mathbf{r}_0 \rrbracket, \quad (4.5)$$

$$\llbracket \mathbf{z}_i \rrbracket \leftarrow \llbracket \mathbf{r}_i \rrbracket - \pi_{i-1}(\llbracket \mathbf{r}_{i-1} \rrbracket), \text{ for } i = 1, \dots, n, \quad (4.6)$$

$$\llbracket \Delta \rrbracket \leftarrow \llbracket \pi_n(\pi_{n-1}(\dots \pi_1(\mathbf{z}_0 + \mathbf{z}_1) \dots + \mathbf{z}_{n-1}) + \mathbf{z}_n) \rrbracket = \llbracket \pi_n(\mathbf{r}_n) \rrbracket, \quad (4.7)$$

where π_0 is the identity permutation. The case of the malicious setting is similar, with each $\llbracket \cdot \rrbracket$ replaced by $\langle \cdot \rangle$. This gives a clean expression of Δ that only depends on π_n and $\mathbf{z}_n$, which allows us to parallelize the invocations to Π_{perm} .

Hence in the offline phase, we only need to invoke Π_{perm} n times in parallel to compute each $\pi_i(\mathbf{r}_i)$. The remaining part of computing $\mathbf{z}_i$ and Δ only involves MPC additions and subtractions, which are communication-free in most secret sharing schemes.

4.3 Checking the Hidden Relation Efficiently

As will be discussed later, the core of malicious security lies in enforcing Equation 4.3. To check it, i.e. to check

$$\beta \mathbf{y}_i = \hat{\mathbf{y}}_i + \mathbf{s}_i,$$

the by-definition approach is to let P_i broadcast $\mathbf{y}_i$ and $\hat{\mathbf{y}}_i$ (instead of sending them to P_{i+1} privately), and all parties jointly compute the difference with MPC operations. Under the $\mathcal{F}_{\text{MPC}}$ -hybrid model, this approach is IT-secure, with $O(n^2m)$ online communication and computation.

To reduce the communication and computation, consider letting P_{i+1} sample a uniform challenge λ_{i+1} , and then broadcasting three elements λ_{i+1} , u_{i+1} , and v_{i+1} , where

$$\langle \beta \rangle \underbrace{\sum_{j=1}^m \lambda_{i+1}^{j-1} \mathbf{y}_i(j)}_{=u_{i+1}} = \underbrace{\sum_{j=1}^m \lambda_{i+1}^{j-1} \hat{\mathbf{y}}_i(j)}_{=v_{i+1}} + \underbrace{\sum_{j=1}^m \lambda_{i+1}^{j-1} \langle \mathbf{s}_i(j) \rangle}_{=w_{i+1}}. \quad (4.8)$$

This way, all parties only need to jointly compute $\langle \beta \rangle u_{i+1} - v_{i+1} - \langle w_{i+1} \rangle$ via MPC operations, which only involves $O(n^2)$ communication with $O(n^2m)$ computation. If λ_{i+1} is uniformly sampled, and Equation 4.8 holds, then by the Schwartz-Zippel lemma, Equation 4.3 holds with probability at least $1 - \frac{m}{|\mathbb{F}|}$.

To further reduce the online computation, as well as to defend against a malicious P_{i+1} who might choose λ_{i+1} maliciously, we can instead generate $\langle \lambda_{i+1} \rangle$ with Π_{rand} and compute $\langle w_{i+1} \rangle$ in the offline phase. The online phase then involves only publicly opening λ_{i+1} and w_{i+1} , and P_{i+1} broadcasting u_{i+1} and v_{i+1} . This involves only $O(n^2 + nm)$ online communication and computation, which meets our efficiency goal.

4.4 Intuition behind Security

To argue the security against malicious adversaries, we wish to reduce the adversary's capabilities to that of a semi-honest one, by proving that either the adversary always sends correct messages to honest parties, i.e. messages that are in accordance with the protocol specification, or the protocol aborts with high probability. This includes the vector $\mathbf{y}_i$ and $\hat{\mathbf{y}}_i$ received by honest P_{i+1} and the u_{i+1} and v_{i+1} broadcast by a malicious party. Note that if both P_i and P_{i+1} are malicious, the private message $\mathbf{y}_i$ and $\hat{\mathbf{y}}_i$ sent from P_i to P_{i+1} are invisible to honest parties, and hence we cannot (and need not) enforce any correctness over them.

The key observation is the following: if the protocol hasn't yet aborted, the malicious adversary must *know* the correct messages it should send to honest parties.⁵

Now, suppose a malicious party P_i knows the correct u_i and v_i to broadcast, and the correct $\mathbf{y}_i$ and $\hat{\mathbf{y}}_i$ that satisfy Equation 4.3. If malicious P_i wants to forge another pair u'_i and v'_i , as it has no control over $\langle w_i \rangle$, it must have known β , as

$$\beta = \frac{v - v'}{u - u'}. \quad (4.9)$$

Likewise, if the adversary wishes to forge another pair $\mathbf{y}'_i$ and $\hat{\mathbf{y}}'_i$, it must have known β , as

$$\beta = \frac{\hat{\mathbf{y}}_i(j) - \hat{\mathbf{y}}'_i(j)}{\mathbf{y}_i(j) - \mathbf{y}'_i(j)}, \quad (4.10)$$

for any entry j where $\mathbf{y}_i(j) \neq \mathbf{y}'_i(j)$.

Hence, due to the knowledge of correct messages, attempting to forge any message that deviates from the protocol is equivalent to trying to guess β correctly, which leads to an abort with high probability. Yet if the adversary sends the correct messages to honest parties, it then obtains the knowledge of the next correct messages. This roughly reduces the security proof to proving the secrecy of β and the security against a semi-honest adversary, which is much more intuitive and easier to handle.

⁵ Although this may sound like an “assumption of knowledge” that tends to be non-falsifiable, we prove in Section C that this statement holds in our case.

5 Semi-honest Secure Shuffle

5.1 Functionality

In this section, we present our semi-honest shuffle protocol. Recall that we assume an ideal functionality $\mathcal{F}_{\text{MPC}}$, which supports the operation

$$\llbracket \pi(\mathbf{x}) \rrbracket \leftarrow \Pi_{\text{perm}}(P_i : \pi, \llbracket \mathbf{x} \rrbracket),$$

where $\mathbf{x}$ is an array of length m , and $\pi : [m] \rightarrow [m]$ is a permutation known only to P_i .

We give a two-phase shuffle protocol, consisting of an offline phase $\text{Shuffle}_{\text{off}}$ and an online phase $\text{Shuffle}_{\text{on}}$. The offline phase is essentially shuffling random values in order to generate a shuffle correlation. The online phase takes as input a length- m array $\llbracket \mathbf{x} \rrbracket$, consumes a fresh shuffle correlation, and outputs shuffled $\llbracket \pi(\mathbf{x}) \rrbracket$.

5.2 Semi-honest Shuffle Correlation

Our shuffle correlation for semi-honest multiparty computation is defined as follows.

Definition 3 (Semi-honest Shuffle Correlation). *The shuffle correlation for semi-honest setting is defined as*

$$\text{cor} := \{(\pi_1, \dots, \pi_n), \llbracket \mathbf{r} \rrbracket, \llbracket \mathbf{s} \rrbracket, (\emptyset, \mathbf{z}_2, \mathbf{z}_3, \dots, \mathbf{z}_n)\},$$

where

1. π_i is an m -permutation (i.e. a permutation on m elements) known only to party P_i . π_i is sampled uniformly from all m -permutations.
2. $\llbracket \mathbf{r} \rrbracket$ and $\llbracket \mathbf{s} \rrbracket$ are two secret-shared random vectors uniform over $\mathbb{F}^m$.
3. $\mathbf{z}_i$ is a random vector of length m and is known only to party P_i . The vectors $\mathbf{z}_i$ are sampled uniformly random, subject to the constraint

$$\mathbf{s} = \pi_n(\dots \pi_3(\pi_2(\pi_1(\mathbf{r}) - \mathbf{z}_2) - \mathbf{z}_3) \dots - \mathbf{z}_n).$$

5.3 Offline Phase

As is discussed in Section 4, we could decouple the correlation between $\mathbf{r}$ and $\mathbf{s}$, by letting

$$\begin{aligned} \llbracket \mathbf{r}_i \rrbracket &\leftarrow \Pi_{\text{rand}}(), \\ \llbracket \mathbf{z}_i \rrbracket &\leftarrow \pi_{i-1}(\llbracket \mathbf{r}_{i-1} \rrbracket) - \llbracket \mathbf{r}_i \rrbracket, \text{ for } i = 2, 3, \dots, n, \\ \llbracket \mathbf{s} \rrbracket &\leftarrow \llbracket \pi_n(\dots \pi_3(\pi_2(\pi_1(\mathbf{r}_1) - \mathbf{z}_2) - \mathbf{z}_3) \dots - \mathbf{z}_n) \rrbracket = \llbracket \pi_n(\mathbf{r}_n) \rrbracket. \end{aligned}$$

One can rigorously show that these two definitions are equivalent, by constructing a bijection and showing their distributions are the same. This definition also offers an immediate solution for generating the shuffle correlation, which is formally presented in Algorithm 1.

Algorithm 1 $\text{cor} \leftarrow \text{Shuffle}_{\text{off}}(m)$

Ensure: Outputs a new shuffle correlation.

Each P_i samples π_i uniformly from all m -permutations.

for $i = 1$ to n **do parallel**

 Generate $\llbracket \mathbf{r}_i \rrbracket$ of length m with Π_{rand} .

$\llbracket \pi_i(\mathbf{r}_i) \rrbracket \leftarrow \Pi_{\text{perm}}(P_i : \pi_i, \llbracket \mathbf{r}_i \rrbracket)$.

end for

for $i = 2$ to n **do parallel**

$\llbracket \mathbf{z}_i \rrbracket \leftarrow \llbracket \pi_{i-1}(\mathbf{r}_{i-1}) \rrbracket - \llbracket \mathbf{r}_i \rrbracket$

 Open $\llbracket \mathbf{z}_i \rrbracket$ to P_i .

end for

Output $\{(\pi_1, \dots, \pi_n), \llbracket \mathbf{r}_1 \rrbracket, \llbracket \pi_n(\mathbf{r}_n) \rrbracket, (\emptyset, \mathbf{z}_2, \mathbf{z}_3, \dots, \mathbf{z}_n)\}$.

5.4 Online Phase

As discussed in Section 4, in the online phase, each party P_i computes and sends to P_{i+1}

$$\mathbf{y}_i \leftarrow \pi_i(\mathbf{y}_{i-1} + \mathbf{z}_i),$$

where $\mathbf{y}_0 = \mathbf{0}$ and $\mathbf{z}_1 := \mathbf{x} - \mathbf{r}_1$ is opened to P_1 . The last party P_n broadcasts $\mathbf{y}_n$, and all parties jointly remove the accumulated mask $-\pi_n(\mathbf{r}_n)$ to obtain the final result. The complete protocol is formalized in Algorithm 2.

Algorithm 2 $\llbracket \pi(\mathbf{x}) \rrbracket \leftarrow \text{Shuffle}_{\text{on}}(\llbracket \mathbf{x} \rrbracket)$

Require: $\{(\pi_1, \dots, \pi_n), \llbracket \mathbf{r}_1 \rrbracket, \llbracket \pi_n(\mathbf{r}_n) \rrbracket, (\emptyset, \mathbf{z}_2, \mathbf{z}_3, \dots, \mathbf{z}_n)\}$ from offline phase.

Ensure: Outputs $\llbracket \pi(\mathbf{x}) \rrbracket$, with $\pi = \pi_n \circ \pi_{n-1} \circ \dots \circ \pi_1$.

$\llbracket \mathbf{z}_1 \rrbracket \leftarrow \llbracket \mathbf{x} \rrbracket - \llbracket \mathbf{r}_1 \rrbracket$

Open $\mathbf{z}_1$ to P_1 .

P_1 computes locally $\mathbf{y}_1 \leftarrow \pi_1(\mathbf{z}_1)$.

P_1 sends $\mathbf{y}_1$ to P_2 .

for $i = 2$ to $n - 1$ **do**

P_i receives $\mathbf{y}_{i-1}$ from P_{i-1} .

P_i computes locally $\mathbf{y}_i \leftarrow \pi_i(\mathbf{z}_i + \mathbf{y}_{i-1})$.

P_i sends $\mathbf{y}_i$ to P_{i+1} .

end for

P_n receives $\mathbf{y}_{n-1}$ from P_{n-1} .

P_n computes locally $\mathbf{y}_n \leftarrow \pi_n(\mathbf{z}_n + \mathbf{y}_{n-1})$.

P_n broadcasts $\mathbf{y}_n$ to all parties.

$\llbracket \mathbf{x}' \rrbracket \leftarrow \mathbf{y}_n + \llbracket \pi_n(\mathbf{r}_n) \rrbracket$

Output $\llbracket \mathbf{x}' \rrbracket$.

To verify the correctness, note that each party P_i holds and sends

$$\begin{aligned}
\mathbf{z}_1 &:= \mathbf{x} - \mathbf{r}_1, & \mathbf{y}_1 &:= \pi_1(\mathbf{z}_1) = \pi_1(\mathbf{x}) - \pi_1(\mathbf{r}_1), \\
\mathbf{z}_2 &:= \pi_1(\mathbf{r}_1) - \mathbf{r}_2, & \mathbf{y}_2 &:= \pi_2(\mathbf{z}_2 + \mathbf{y}_1) = \bar{\pi}_2(\mathbf{x}) - \pi_2(\mathbf{r}_2), \\
\mathbf{z}_3 &:= \pi_2(\mathbf{r}_2) - \mathbf{r}_3, & \mathbf{y}_3 &:= \pi_3(\mathbf{z}_3 + \mathbf{y}_2) = \bar{\pi}_3(\mathbf{x}) - \pi_3(\mathbf{r}_3), \\
& & \vdots & \\
\mathbf{z}_n &:= \pi_{n-1}(\mathbf{r}_{n-1}) - \mathbf{r}_n, & \mathbf{y}_n &:= \pi_n(\mathbf{z}_n + \mathbf{y}_{n-1}) = \bar{\pi}_n(\mathbf{x}) - \pi_n(\mathbf{r}_n),
\end{aligned}$$

where $\bar{\pi}_i$ is an abbreviation for $\pi_i \circ \pi_{i-1} \circ \dots \circ \pi_1$. Hence, it is straightforward that the final result is

$$\llbracket \pi(\mathbf{x}) \rrbracket = \mathbf{y}_n + \llbracket \pi_n(\mathbf{r}_n) \rrbracket.$$

Due to page limits, the security proof is provided in Section B of the appendix.

6 Maliciously Secure Shuffle

As discussed in Section 4, our malicious shuffle protocol requires an additional MAC and a checking procedure. Hence in this section, we first define the malicious shuffle correlation, then present the MAC checking protocols, and finally represent the offline and online phases of the malicious shuffle protocol.

6.1 Malicious Shuffle Correlation

The shuffle correlation for malicious multiparty computation is defined as follows.

Definition 4 (Extended Malicious Shuffle Correlation). *The shuffle correlation for malicious setting is defined as*

$$\text{cor} = \begin{pmatrix} \langle \beta \rangle & \langle \mathbf{r} \rangle & \langle \beta \mathbf{r} \rangle & \langle \pi_1^{-1}(\mathbf{r}'_1) \rangle & \langle \mathbf{s} \rangle \\ \pi_1 & \pi_2 & \pi_3 & \pi_4 & \pi_5 & \dots & \pi_n \\ \langle \mathbf{r}'_1 \rangle & \langle \mathbf{r}'_2 \rangle & \langle \mathbf{r}'_3 \rangle & \langle \mathbf{r}'_4 \rangle & \langle \mathbf{r}'_5 \rangle & \dots & \langle \mathbf{r}'_n \rangle \\ \langle c_2 \rangle & \langle c_3 \rangle & \langle c_4 \rangle & \langle c_5 \rangle & \dots & \langle c_n \rangle \\ \langle d_2 \rangle & \langle d_3 \rangle & \langle d_4 \rangle & \langle d_5 \rangle & \dots & \langle d_n \rangle \\ \mathbf{z}_2 & \mathbf{z}_3 & \mathbf{z}_4 & \mathbf{z}_5 & \dots & \mathbf{z}_n \end{pmatrix},$$

where

1. π_i is an m -permutation known only to P_i . It is sampled uniformly from all possible m -permutations by P_i (if P_i is honest).
2. $\langle \beta \rangle$ is a secret-shared random variable. It plays the role of the authentication key, i.e. MAC key.
3. $\langle \mathbf{r} \rangle, \langle \mathbf{s} \rangle, \langle \mathbf{r}'_1 \rangle, \dots, \langle \mathbf{r}'_n \rangle$ are secret-shared vectors of length m , with each entry independently uniformly random. $\langle \beta \mathbf{r} \rangle$ is secret-shared $\beta \cdot \mathbf{r}$.

4. c_i is uniformly random, with

$$d_i = \sum_{j=1}^m c_i^{j-1} \mathbf{r}'_{i-1}(j).$$

5. $\mathbf{z}_i = (\mathbf{z}_i^1, \mathbf{z}_i^2)$, where each $\mathbf{z}_i^j$ is a vector of length m . The entries of $\mathbf{z}_i^1$ are uniformly random, under the constraint

$$\begin{cases} \mathbf{z}_i^2 = \beta \mathbf{z}_i^1 + \mathbf{r}'_{i-1} - \pi_i^{-1}(\mathbf{r}'_i) & \forall i = 2, 3, \dots, n \\ \mathbf{s} = \pi_n(\pi_{n-1}(\dots \pi_2(\pi_1(\mathbf{r}) - \mathbf{z}_2^1) - \mathbf{z}_3^1 \dots) - \mathbf{z}_n^1) \end{cases}$$

Remark. For readers familiar with SPDZ [14], we note that though β plays a role that is similar to the MAC key in SPDZ framework (which is usually denoted as “ α ” in context), they are different and should not be confused together. For example, when instantiated with SPDZ, the maliciously securely shared (say) $\langle \mathbf{r} \rangle$ is essentially $(\llbracket \mathbf{r} \rrbracket, \llbracket \alpha \mathbf{r} \rrbracket)$, i.e. the value and its SPDZ MAC. Our MAC for shuffle is then $\langle \beta \mathbf{r} \rangle = (\llbracket \beta \mathbf{r} \rrbracket, \llbracket \alpha \beta \mathbf{r} \rrbracket)$, which is orthogonal and additional to the SPDZ MAC $\llbracket \alpha \mathbf{r} \rrbracket$.

6.2 Correlation Check of Public Values

The functionality of protocol $\text{Verify}(\mathbf{a}, \mathbf{b}, \langle \beta \rangle, \langle \mathbf{r} \rangle)$ is to check publicly whether $\beta \mathbf{a} = \mathbf{b} + \mathbf{r}$, where both $\mathbf{a}$ and $\mathbf{b}$ are publicly known. This becomes helpful when in the online phase of the shuffle protocol, parties need to check whether P_n has broadcast corrected $\mathbf{y}_n^1$ and $\mathbf{y}_n^2$.

Such a check can be done easily by opening

$$\mathbf{a} \langle \beta \rangle - \mathbf{b} - \langle \mathbf{r} \rangle$$

and checking if it is zero. As many MPC frameworks are based on linear secret sharing scheme, it is likely that the overhead of this protocol amounts to a single Π_{open} , as the addition and the multiplication (with constant) are done locally.

This protocol is formally presented in Algorithm 3.

Algorithm 3 $\text{Verify}(\mathbf{a}, \mathbf{b}, \langle \beta \rangle, \langle \mathbf{r} \rangle)$

Require: By protocol design, $\beta \mathbf{a} = \mathbf{b} + \mathbf{r}$.

Ensure: Abort if $\beta \mathbf{a} \neq \mathbf{b} + \mathbf{r}$.

 Compute $\mathbf{d} \leftarrow \mathbf{a} \langle \beta \rangle - \mathbf{b} - \langle \mathbf{r} \rangle$.

 Open $\mathbf{d}$ for all parties.

 If $\mathbf{d} \neq \mathbf{0}$, abort.

To see the security of this protocol, note that every operation carried out in the protocol is via $\mathcal{F}_{\text{MPC}}$, i.e. some trivial arithmetic operations plus a public opening operation. Hence, the protocol is secure by definition.

Note that protocol Verifier requires $O(nm)$ communication, which is not feasible to be called for each party. Hence in later implementation, we invoke this protocol only once, over the output of P_n . However, to ensure the integrity of each intermediate $\mathbf{y}_i$ sent to honest P_{i+1} , we need a more efficient variant of this protocol, which we call PartialVerify and is formally presented in Algorithm 4.

Algorithm 4 PartialVerify($P_{i+1}, \mathbf{y}_i^1, \mathbf{y}_i^2, \langle \beta \rangle, \langle c_{i+1} \rangle, \langle d_{i+1} \rangle$)

Require: $\langle c_{i+1} \rangle$ and $\langle d_{i+1} \rangle$ come from malicious shuffle correlation.

Ensure: If honest P_{i+1} does not receive $\beta \mathbf{y}_i^1 = \mathbf{y}_i^2 + \pi_i(\mathbf{r}'_i)$, abort w.h.p.

Parties open $\langle c_{i+1} \rangle$ to P_{i+1} .

P_{i+1} computes $w_b := \sum_{j=1}^m c_{i+1}^{j-1} \mathbf{y}_i^b(j)$, $b \in \{1, 2\}$.

P_{i+1} broadcasts w_1 and w_2 .

$\langle e \rangle \leftarrow w_1 \cdot \langle \beta \rangle - w_2 - \langle d_{i+1} \rangle$.

Open e for all parties.

Abort if $e \neq 0$.

6.3 Offline Phase

Recall that we assume $\mathcal{F}_{\text{MPC}}$ supports command Π_{perm} , such that

$$(\langle \pi(\mathbf{x}_1) \rangle, \dots, \langle \pi(\mathbf{x}_t) \rangle) \leftarrow \Pi_{\text{perm}}(P_i : \pi, \langle \mathbf{x}_1 \rangle, \dots, \langle \mathbf{x}_t \rangle).$$

The offline phase is formally described in Algorithm 5.

Remark. Computing $\langle c_i^j \rangle$ for all $j = 2, 3, \dots, m-1$ can be done efficiently with a divide-and-conquer strategy, costing $O(m)$ calls to Π_{mul} and $O(\log m)$ rounds. The round complexity can be further reduced to $O(1)$ if given access to sampling random $\langle u \rangle$ and $\langle u^{-1} \rangle$, as shown in [12].

Alternatively, we can also omit $\langle c_i^j \rangle$ and $\langle d_i \rangle$ from the correlation, and generate them after opening c_i in the online phase. This approach requires $O(n^2 + nm)$ online communication and $O(n^2 m)$ online computation.

To understand the process, note that the online phase consists essentially of two semi-honest shuffle protocols, one for shuffling $\mathbf{x}$ and the other for $\beta \mathbf{x}$. Naturally, the mask for shuffling $\mathbf{x}$ is the same as in the semi-honest case, which is

$$\mathbf{z}_i^1 = \pi_{i-1}(\mathbf{r}_{i-1}) - \mathbf{r}_i. \quad (6.1)$$

As for the mask for $\beta \mathbf{x}$, we need to ensure that

$$\beta \mathbf{y}_i^1 = \mathbf{y}_i^2 + \mathbf{r}'_i, \quad (6.2)$$

Algorithm 5 $\text{cor} \leftarrow \text{Shuffle}_{\text{off}}(m)$

Ensure: Outputs a shuffle correlation.

Each party P_i samples m -permutation π_i uniformly random.

Generate random value $\langle \beta \rangle$.

for $i = 1$ to n **do parallel**

Generate random vectors $\langle \mathbf{r}_i \rangle$ and $\langle \mathbf{r}'_i \rangle$.

 $\langle \beta \mathbf{r}_i \rangle \leftarrow \Pi_{\text{mul}}(\langle \mathbf{r}_i \rangle, \langle \beta \rangle)$
 $\langle \pi_i(\mathbf{r}_i), \pi_i(\mathbf{r}'_i) \rangle \leftarrow \Pi_{\text{perm}}(P_i : \pi_i, \langle \mathbf{r}_i, \mathbf{r}'_i \rangle)$
if $i \geq 2$ **then**
 $\langle \mathbf{z}_i^1 \rangle \leftarrow \langle \pi_{i-1}(\mathbf{r}_{i-1}) \rangle - \langle \mathbf{r}_i \rangle$
 $\langle \mathbf{z}_i^2 \rangle \leftarrow \langle \pi_{i-1}(\mathbf{r}'_{i-1}) \rangle - \langle \mathbf{r}'_i \rangle + \Pi_{\text{mul}}(\langle \mathbf{z}_i^1 \rangle, \langle \beta \rangle)$
 $\langle \mathbf{z}_i \rangle := (\langle \mathbf{z}_i^1 \rangle, \langle \mathbf{z}_i^2 \rangle)$
 $\langle c_i \rangle \leftarrow \Pi_{\text{rand}}()$

Compute $\langle c_i \rangle, \langle c_i^2 \rangle, \dots, \langle c_i^{m-1} \rangle$.

 $\triangleright O(m)$ calls to Π_{mul} .

 $\langle d_i \rangle \leftarrow \sum_{j=1}^m \langle c_i^{j-1} \rangle \cdot \langle \pi_{i-1}(\mathbf{r}'_{i-1}(j)) \rangle$
end if
end for
for $i = 2$ to n **do parallel**

Open $\mathbf{z}_i$ to P_i .

end for

$$\text{Output cor} = \begin{pmatrix} \langle \beta \rangle & \langle \mathbf{r}_1 \rangle & \langle \beta \mathbf{r}_1 \rangle & \langle \mathbf{r}'_1 \rangle & \langle \pi_n(\mathbf{r}_n) \rangle \\ \pi_1 & \pi_2 & \pi_3 & \cdots & \pi_n \\ \langle \pi_1(\mathbf{r}'_1) \rangle & \langle \pi_2(\mathbf{r}'_2) \rangle & \langle \pi_3(\mathbf{r}'_3) \rangle & \cdots & \langle \pi_n(\mathbf{r}'_n) \rangle \\ & \langle c_2 \rangle & \langle c_3 \rangle & \cdots & \langle c_n \rangle \\ & \langle d_2 \rangle & \langle d_3 \rangle & \cdots & \langle d_n \rangle \\ & \mathbf{z}_2 & \mathbf{z}_3 & \cdots & \mathbf{z}_n \end{pmatrix}$$

which is consistent with the integrity check to be carried out in the online phase. Further, note that

$$\mathbf{y}_i^1 = \pi_i(\mathbf{y}_{i-1}^1 + \mathbf{z}_i^1), \quad (6.3)$$

$$\mathbf{y}_i^2 = \pi_i(\mathbf{y}_{i-1}^2 + \mathbf{z}_i^2) = \pi_i(\beta \mathbf{y}_{i-1}^1 - \mathbf{r}'_{i-1} + \mathbf{z}_i^2). \quad (6.4)$$

By plugging the Equations 6.3 and 6.4 into Equation 6.2, we conclude that

$$\beta \pi_i(\mathbf{y}_{i-1}^1 + \mathbf{z}_i^1) = \pi_i(\beta \mathbf{y}_{i-1}^1 - \mathbf{r}'_{i-1} + \mathbf{z}_i^2) + \mathbf{r}'_i \quad (6.5)$$

$$\implies \beta \mathbf{y}_{i-1}^1 + \beta \mathbf{z}_i^1 = \beta \mathbf{y}_{i-1}^1 - \mathbf{r}'_{i-1} + \mathbf{z}_i^2 + \pi_i^{-1}(\mathbf{r}'_i) \quad (6.6)$$

$$\implies \mathbf{z}_i^2 = \beta \mathbf{z}_i^1 + \mathbf{r}'_{i-1} - \pi_i^{-1}(\mathbf{r}'_i). \quad (6.7)$$

To avoid computing $\pi_i^{-1}(\mathbf{r}'_i)$, we simply replace each $\mathbf{r}'_i$ term by $\pi_i(\mathbf{r}'_i)$, which finally leads to

$$\mathbf{z}_i^2 = \beta \mathbf{z}_i^1 + \pi_{i-1}(\mathbf{r}'_{i-1}) - \mathbf{r}'_i. \quad (6.8)$$

This explains the process formalized in Algorithm 5.

Similar to the semi-honest case, it is also straightforward to show that the output of Algorithm 5 is accordant with Definition 4.

6.4 Online Shuffle

As also mentioned in the former section, the online phase consists essentially of two concurrent sessions of semi-honest shuffle, i.e. each P_i sends to P_{i+1}

$$\begin{aligned} \mathbf{y}_i^1 &\leftarrow \pi_i(\mathbf{y}_{i-1}^1 + \mathbf{z}_i^1) = \bar{\pi}_i(\mathbf{x}) - \pi_i(\mathbf{r}_i), \\ \mathbf{y}_i^2 &\leftarrow \pi_i(\mathbf{y}_{i-1}^2 + \mathbf{z}_i^2) = \bar{\pi}_i(\beta\mathbf{x}) - \pi_i(\beta\mathbf{r}_i + \mathbf{r}'_i). \end{aligned}$$

We abbreviate this operation as $\mathbf{y}_i \leftarrow \pi_i(\mathbf{y}_{i-1} + \mathbf{z}_i)$, where each vector is of length $2m$. Applying π_i means permuting both halves of the $2m$ -length vector by π_i .

To achieve malicious security, parties check the integrity of each $\mathbf{y}_i$, by calling

$$\text{PartialVerify}(P_{i+1}, \mathbf{y}_i^1, \mathbf{y}_i^2, \langle\beta\rangle, \langle c_{i+1}\rangle, \langle d_{i+1}\rangle).$$

Note that for honest P_{i+1} , this test can pass only if it receives message with correct correlation $\beta\mathbf{y}_i^1 = \mathbf{y}_i^2 + \mathbf{r}'_i$, which is crucial for establishing the protocol's security.

The protocol is formally presented in Algorithm 6.

Algorithm 6 $\langle\bar{\pi}_n(\mathbf{x})\rangle \leftarrow \text{Shuffle}_{\text{on}}(\langle\mathbf{x}\rangle)$

Require: Fresh cor from Algorithm 5.

Ensure: Outputs $\langle\bar{\pi}_n(\mathbf{x})\rangle$ if the protocol does not abort.

$\langle\beta\mathbf{x}\rangle \leftarrow \Pi_{\text{mul}}(\langle\beta\rangle, \langle\mathbf{x}\rangle)$
 $\langle\mathbf{z}_1^1\rangle \leftarrow \langle\mathbf{x}\rangle - \langle\mathbf{r}_1\rangle$
 $\langle\mathbf{z}_1^2\rangle \leftarrow \langle\beta\mathbf{x}\rangle - \langle\beta\mathbf{r}_1\rangle - \langle\mathbf{r}'_1\rangle$
 Open $\mathbf{z}_1 := (\mathbf{z}_1^1, \mathbf{z}_1^2)$ to P_1 .
 P_1 computes and broadcasts $\mathbf{y}_1 = \pi_1(\mathbf{z}_1)$.
for $i = 2$ to $n - 1$ **do**
 P_i computes and sends to P_{i+1} $\mathbf{y}_i = \pi_i(\mathbf{z}_i + \mathbf{y}_{i-1})$.
end for
 P_n computes and broadcasts $\mathbf{y}_n = \pi_n(\mathbf{z}_n + \mathbf{y}_{n-1})$.
for $i = 1$ to $n - 1$ **do**
 $\text{PartialVerify}(P_{i+1}, \mathbf{y}_i^1, \mathbf{y}_i^2, \langle\beta\rangle, \langle c_{i+1}\rangle, \langle d_{i+1}\rangle)$
end for
 Verify($\mathbf{y}_n^1, \mathbf{y}_n^2, \langle\beta\rangle, \langle\pi_n(\mathbf{r}'_n)\rangle$)
 $\langle\mathbf{y}_n^1\rangle \leftarrow \Pi_{\text{input}}(\mathbf{y}_n^1)$.
 $\langle\mathbf{x}'\rangle \leftarrow \langle\mathbf{y}_n^1\rangle + \langle\pi_n(\mathbf{r}_n)\rangle$
 Output $\langle\mathbf{x}'\rangle$.

To see the correctness of above process when all parties are honest, note that each P_i holds $\mathbf{z}_i$, where

$$\begin{aligned} \mathbf{z}_1 &= \mathbf{x} - \mathbf{r}_1, & \beta\mathbf{x} - \beta\mathbf{r}_1 - \mathbf{r}'_1, \\ \mathbf{z}_2 &= \pi_1(\mathbf{r}_1) - \mathbf{r}_2, & \pi_1(\beta\mathbf{r}_1 + \mathbf{r}'_1) - \beta\mathbf{r}_2 - \mathbf{r}'_2, \\ &\vdots & \\ \mathbf{z}_n &= \pi_{n-1}(\mathbf{r}_{n-1}) - \mathbf{r}_n, & \pi_{n-1}(\beta\mathbf{r}_{n-1} + \mathbf{r}'_{n-1}) - \beta\mathbf{r}_n - \mathbf{r}'_n. \end{aligned}$$

And the $\mathbf{y}_i$ sent by each party P_i to P_{i+1} is

$$\begin{aligned} \mathbf{y}_1 &= \pi_1(\mathbf{x}) - \pi_1(\mathbf{r}_1), \pi_1(\beta\mathbf{x}) - \pi_1(\beta\mathbf{r}_1 + \mathbf{r}'_1), \\ \mathbf{y}_2 &= \pi_2(\mathbf{x}) - \pi_2(\mathbf{r}_2), \pi_2(\beta\mathbf{x}) - \pi_2(\beta\mathbf{r}_2 + \mathbf{r}'_2), \\ &\vdots \\ \mathbf{y}_n &= \pi_n(\mathbf{x}) - \pi_n(\mathbf{r}_n), \pi_n(\beta\mathbf{x}) - \pi_n(\beta\mathbf{r}_n + \mathbf{r}'_n). \end{aligned}$$

Hence, if the parties input the first m entries of $\mathbf{y}_n$ as $\langle \mathbf{y}_n^1 \rangle$ and add it by $\langle \pi_n(\mathbf{r}_n) \rangle$, the result is indeed $\langle \pi_n(\mathbf{x}) \rangle$. Also, if all parties act honestly, the calls to PartialVerify and Verify should all pass.

Remark. We note that in real-world MPC implementations, ensuring the correctness of Π_{open} usually requires the parties to perform an additional consistency check before opening. For instance, in the SPDZ framework [14], the parties execute such a check before revealing the final output and abort if it fails. This guarantees that the opening procedure either produces the correct value or aborts without revealing any information. However, prior to the check, a malicious party can arbitrarily manipulate the reconstructed (intermediate) values during the protocol, by changing its local share. Consequently, an opening operation in MPC is generally not secure on its own.

In contrast, this issue does not arise in the $\mathcal{F}_{\text{MPC}}$ -hybrid model, where Π_{open} and all other commands provided by $\mathcal{F}_{\text{MPC}}$ are secure by definition. Therefore, in Algorithm 6, once a corrupted party P_i sends $\mathbf{y}_i$ to honest P_{i+1} , the adversary can no longer influence the computation or the opening of the value d in the following PartialVerify protocol in Algorithm 4.

Thus in Algorithm 6, either P_i sends $\mathbf{y}_i$ with correct correlation

$$\langle \beta \rangle \mathbf{y}_i^1 = \mathbf{y}_i^2 + \langle \pi_i(\mathbf{r}'_i) \rangle,$$

to honest P_{i+1} , or the protocol aborts during the verification step with high probability. This defends the protocol against the selective abort attack in [35] where the malicious P_i tampers $\mathbf{y}_i$ trying to learn the π_{i+1} chosen by honest P_{i+1} , and is crucial for establishing the security of our protocols.

We will further discuss this gap in Section E, along with possible ways to address it in real-world implementations. The security proof under $\mathcal{F}_{\text{MPC}}$ -hybrid model is also presented in Appendix C.

6.5 Complexity Analysis

The offline communication and computation complexity of our protocol is $O(nP + nmR + nmM)$, where P is the overheads for Π_{perm} , R is the overheads for Π_{rand} , and M is the overheads for Π_{mul} . If $R = M = O(n)$ (e.g. [17, 18]), since the best-known result gives $P = O(nm \log m)$ (e.g. [28, 35]), the offline complexities are $O(nP)$. This indicates that the offline complexities are asymptotically the same as invoking Π_{perm} for n times. Since these invocations to Π_{perm} are inevitable for the permute-in-turn paradigm, our construction obtains linear online phase for

free. It should be noted though, that as our shuffle protocol invokes Π_{perm} for permuting columns of $\ell \times m$ matrix with $\ell = 3$, the concrete offline communication and computation can be doubled in worst case.⁶

The online communication complexity of our protocol is $O(n^2 + nm)$. By passing around $\mathbf{y}_i$ and finally broadcasting $\mathbf{y}_n$, the communication of all parties is $O(nm)$, since each vector is of length $2m$. The verification requires that parties open c_i to P_i , and that P_i computes and broadcasts two elements w_1 and w_2 , which is $O(n)$ communication per party. Also, each check requires opening one shared element, which is in total $O(n^2)$ communication. Summing up, this is $O(n^2 + nm)$. Since in practice, the number of items to be shuffled is usually much larger than the number of participants, this is $O(nm)$ in most cases.

The online computation complexity of our protocol is also $O(n^2 + nm)$, measured by field operations. After receiving $\mathbf{y}_{i-1}$, party P_i needs to compute w_1 , w_2 and $\mathbf{y}_i$, each of which requires $O(m)$ computation. This is hence $O(n^2 + nm)$ for all parties.

6.6 Intuition behind Security

Due to page limitation, we defer the full security proof to Appendix C. Here we provide a proof sketch to offer intuition. We note that, the security of input $\mathbf{x}$ is rather straightforward, as it is only used in the online phase, and is hidden behind fresh random masks. The main challenge is to show that, if the protocol does not abort, then the secret permutation π_i of honest party P_i is protected. Note again that as mentioned earlier, we do not need to protect π_i if the protocol aborts, since it's not an input for shuffle protocol.

Theorem 1 (Sketch). *The final protocol achieves UC-security under $\mathcal{F}_{\text{MPC}}$ -hybrid model. That is, there exists a UC simulator $\mathcal{S}$, such that the environment $\mathcal{E}$ cannot distinguish whether it's interacting with the adversary $\mathcal{A}$ or with simulator $\mathcal{S}$, i.e.*

$$|\Pr[1 \leftarrow (\mathcal{E} \bowtie \Pi_{T,\mathcal{A}})] - \Pr[1 \leftarrow (\mathcal{E} \bowtie \mathcal{F}_{T,\mathcal{S}})]| < \epsilon = O(2^{-\kappa}).$$

Proof (Sketch). Assuming the semi-honest shuffle protocol is secure, we now show how the simulation of malicious shuffle protocol can be reduced to simulation of semi-honest case.

Let P_i be the first corrupted party that behaves maliciously and sends wrong message to honest parties. This means P_i either (1) sends wrong message to *honest* P_{i+1} , or (2) broadcasts wrong w_1 or w_2 in `PartialVerify`. (Note that if P_{i+1} is also malicious, the message P_i sends to P_{i+1} *in private* is irrelevant to the protocol's integrity.) Intuitively, the security relies on the following facts/intuitions:

1. The adversary knows the correct $\mathbf{y}_i$ or w_1 and w_2 to send.

⁶ In some constructions of Π_{perm} (e.g. [35]), the communication is $O(nm \log m + nm\ell)$. Hence, the actual growth of offline communication could be even milder, if $nm \log m$ term dominates.

2. The adversary cannot learn β if the protocol hasn't aborted.
3. The protocol PartialVerify aborts if honest P_{i+1} receives $\beta \mathbf{y}_i^1 \neq \mathbf{y}_i^2 + \mathbf{r}_i'$.

The first statement holds, because P_i is the first one to act maliciously. The second holds, because $\langle \beta \rangle$ is never opened without independent random mask. The third holds, because the security of PartialVerify can be roughly reduced to Verify, which is perfectly secure in $\mathcal{F}_{\text{MPC}}$ -hybrid model.

Now for case (1), suppose P_i sends $\mathbf{t}_i \neq \mathbf{y}_i$ to honest P_{i+1} . If $\mathbf{t}_i^2 \neq \beta \cdot \mathbf{t}_i^1 + \mathbf{r}$, the protocol aborts due to the third fact. If $\mathbf{t}_i^2 = \beta \cdot \mathbf{t}_i^1 + \mathbf{r}$, we have $\mathbf{t}_i^1 \neq \mathbf{y}_i^1$, and since P_i knows correct $\mathbf{y}_i$, it knows

$$\beta = \frac{\mathbf{y}_i^2(j) - \mathbf{t}_i^2(j)}{\mathbf{y}_i^1(j) - \mathbf{t}_i^1(j)},$$

for some j where $\mathbf{y}_i^1(j) \neq \mathbf{t}_i^1(j)$. This contradicts the second fact as P_i now solves β explicitly.

A similar argument applies for case (2). Suppose P_i broadcasts wrong w_1' or w_2' . If $\beta w_1' \neq w_2' + \langle d_i \rangle$, the protocol aborts trivially. If $\beta w_1' = w_2' + \langle d_i \rangle$, since P_i knows correct w_1 and w_2 , P_i knows

$$\beta = \frac{w_2 - w_2'}{w_1 - w_1'},$$

contradicting the second fact.

Thus, the probability of P_i succeeding in sending wrong messages to honest parties without being detected is the same as P_i guessing β correct, which is negligible. Hence, the environment either misbehaves and receives identical views upon protocol aborts (w.h.p. in both worlds), or acts honestly and receives a view for semi-honest adversary. This reduces the malicious security to semi-honest security, which is assumed to hold. $\square$

7 Experiments

7.1 Experimental Setup

We implement our shuffle protocol using the permutation protocol of Song et al. [35] and the MP-SPDZ [25] MPC framework. The MP-SPDZ framework supports the Mascot protocol [26], an arithmetic MPC protocol based on additive secret sharing. Mascot supports the multiplication operations required by our protocol, and is secure against malicious adversaries corrupting a majority of parties. We first implement the permutation protocol and the shuffle protocol of Song et al. (for convenience, we refer to it as the “basic shuffle protocol” in the following), then implement our shuffle protocol based on it, and compare the performance of the two protocols. The goal of our experiment is to evaluate whether our construction significantly improves the online communication and computation complexity compared to the basic shuffle protocol, as suggested by our theoretical analysis.

We note that the choice of Mascot as the specific MPC framework is somewhat arbitrary, as our constructions are also compatible to other frameworks. The point is that we implement the shuffle protocol of Song et al. also for Mascot, so both shuffle protocols are implemented and compared within the same MPC framework.

Our experiment is run on a host equipped with 32 processors, each being Intel(R) Xeon(R) Gold 5122 CPU @ 3.60GHz. The operating system is Ubuntu 18.04.6 LTS. Each party is simulated as a separate process on the host, and the communication is carried out via loopback. We use the Linux `tc` command to simulate a WAN network, with a bandwidth of 80 MB/s and RTT 60 ms.

We set the security parameter to $\kappa = 40$ and choose a prime field of size 2^{64} for the Mascot protocol. The basic shuffle protocol of [35] includes a parameter that trades off between communication and computation overheads, due to the application of sub-permutation decomposition technique of [7]. This parameter also affects offline/online running time. Hence, we report the results of two configurations of the basic protocol: one optimized for minimal online running time and the other for minimal total running time. Since the online phase of our protocol is unaffected by the choice of basic permutation protocol, we report results for the configuration with the minimal overall running time.

Our code is available at <https://anonymous.4open.science/r/MP-SPDZ-Suffle-4F68/>.

7.2 Experiments with Number of Parties

Table 2. Offline/Online Communication and Running Time

	n	Communication per Party (MB)					Running Time (s)				
		3	6	9	12	15	3	6	9	12	15
Offline	[35] ¹	184	493	865	1260	1679	64.3	282	663	1214	2008
	[35] ²	135	355	629	899	1278	102	401	936	1621	2735
	ours	412	1294	2327	3552	4896	103	444	1085	1987	2875
Online	[35] ¹	14.0	35.0	56.1	76.4	92.6	8.65	38.6	76.8	128	187
	[35] ²	7.73	18.3	28.3	36.0	45.9	6.67	25.3	47.8	66.0	95.4
	ours	0.262	0.329	0.353	0.366	0.376	0.625	1.27	1.94	3.41	4.32

n is the number of parties. The number of items is $m = 2^{12}$.

[35]¹ is optimized to minimize total running time.

[35]² is optimized to minimize online running time.

We first evaluate the protocols with $n = 3, 6, 9, 12, 15$ parties. This test aims to verify that our protocol has linear online communication, while the basic shuffle protocol has $O(Bn^2m)$ online communication, where m is the number of items. The results of this experiment are listed in Table 2.

From the table, it can be seen that the growth of both online communication and running time of our protocol appears to be significantly slower than that of

the basic protocol. One significant phenomenon is that the online communication overhead per party is almost invariant for our protocol, which aligns with the theoretical result that the online communication should be $O(n(n+m))$. It can also be seen that the basic protocol’s trade-off has inherent limitations, in the sense that even if we choose a parameter that completely ignores the offline cost, its online running time remains slower than that of our protocol. This is because the basic shuffle protocol has its complexity inherently $\Theta(Bn^2m)$, which cannot be mitigated by increasing offline computation overhead.

It should also be noted that, when optimizing the basic protocol by its total running time, its offline communication and running time are both significantly smaller than ours, by approximately 0.33× and 0.65×, respectively. However, the online communication and running time of our protocol outperform those of the basic protocol by approximately 40×. This aligns with the theoretical analysis that the online complexity of basic protocol should grow quadratically with the number of parties, while our protocol only linearly.

7.3 Experiments with Number of Items

We also test the protocols for shuffling $m = 2^{10}, 2^{12}, \dots, 2^{18}$ items between $n = 2$ parties. These tests aim to evaluate the scalability of the protocol with respect to increasing data volume. The results are listed in Table 3.

Table 3. Offline/Online Communication and Running Time

	$\log_2 m$	Communication per Party (MB)					Running Time (s)				
		10	12	14	16	18	10	12	14	16	18
Offline	[35] ¹	22.4	89.0	449	1770	8473	7.84	30.0	135	553	2586
	[35] ²	23.5	64.8	275	1069	5012	140	91.2	314	2029	3481
	ours	56.5	230	908	2759	11950	15.7	51.7	213	771	3495
Online	[35] ¹	1.88	6.94	35.6	135	666	1.15	1.88	8.66	38.3	159
	[35] ²	0.999	3.40	14.1	49.2	285	0.972	0.972	1.75	10.8	35.2
	ours	0.049	0.196	0.786	3.14	12.5	0.336	0.350	0.398	0.502	1.23

m is the number of items. The number of parties is $n = 2$.

[35]¹ is optimized to minimize total running time.

[35]² is optimized to minimize online running time.

From the table, we can observe that both online communication and running time of our protocol significantly lower than those of the basic protocols. Also, it can be observed that the communication and running times of all three protocols scale approximately by a factor of four, which is consistent with the theoretical result that all the protocols are linear with respect to the number of items. Compared to the basic protocol optimized for minimal total running time, our protocol is approximately 1.4× slower in the offline phase, yet is around 100× faster in the online phase when the number of items is large. This advantage

arises from the factor B introduced by the cut-and-choose technique used in the basic protocol. The performance gap becomes more pronounced as the number of items grows. This is possibly because, when the number of items is small, the computation (rather than communication) remains the primary bottleneck.

8 Conclusion

In this paper, we study the design of an MPC shuffle protocol that minimizes online overhead. We formalize the permute-in-turn paradigm and propose generic transformations from semi-honest (resp. malicious) MPC permutation protocol to semi-honest (resp. malicious) MPC shuffle protocol with linear online communication and computation. Notably, instantiating our constructions with the MPC permutation protocol by Song et al. [35] yields the first maliciously secure MPC shuffle protocol with a linear online phase under additive secret sharing. Likewise, instantiating with the permutation protocol by Keller and Scholl [28], we obtain the first MPC shuffle protocol with a linear online phase for Shamir secret sharing for both semi-honest and malicious security. We present security proofs for both constructions and show that our maliciously secure construction achieves UC security under $\mathcal{F}_{\text{MPC}}$ -hybrid model. Experimental results show that our protocols significantly outperform existing shuffle protocols in both online communication and runtime.

References

1. Abe, M.: Mix-networks on permutation networks. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 258–273. Springer (1999)
2. Adida, B., Wikström, D.: How to shuffle in public. In: Theory of Cryptography: 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21–24, 2007. Proceedings 4. pp. 555–574. Springer (2007). https://doi.org/10.1007/978-3-540-70936-7_30
3. Alexopoulos, N., Kiayias, A., Talviste, R., Zacharias, T.: Mmix: Anonymous messaging via secure multiparty computation. In: 26th USENIX Security Symposium (USENIX Security 17). pp. 1217–1234 (2017)
4. Bater, J., Elliott, G., Eggen, C., Goel, S., Kho, A., Rogers, J.: Smcql: Secure querying for federated databases (2017). <https://doi.org/10.14778/3055330.3055334>
5. Beerliová-Trubíniová, Z., Hirt, M.: Perfectly-secure mpc with linear communication complexity. In: Theory of Cryptography Conference. pp. 213–230. Springer (2008). https://doi.org/10.1007/978-3-540-78524-8_13
6. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: Proceedings 42nd IEEE Symposium on Foundations of Computer Science. pp. 136–145. IEEE (2001). <https://doi.org/10.1109/SFCS.2001.959888>
7. Chase, M., Ghosh, E., Poburinnaya, O.: Secret-shared shuffle. In: Advances in Cryptology–ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part III 26. pp. 342–372. Springer (2020). https://doi.org/10.1007/978-3-030-64840-4_12

8. Chaum, D.L.: Untraceable electronic mail, return addresses, and digital pseudonyms. *Communications of the ACM* **24**(2), 84–90 (1981). <https://doi.org/10.1145/358549.358563>
9. Chen, K., Pang, B., Li, Y., Wang, M.: Secure multi-party shuffling with optimal communication. *The Computer Journal* p. bxaf043 (2025)
10. Chow, S., Lee, J., Subramanian, L.: Two-party computation model for privacy-preserving queries over distributed databases (2009), publisher Copyright: © 2009 Proceedings of the Symposium on Network and Distributed System Security, NDSS 2009
11. Dalskov, A., Escudero, D., Keller, M.: Fantastic four: Honest-Majority Four-Party secure computation with malicious security. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 2183–2200. USENIX Association (Aug 2021), <https://www.usenix.org/conference/usenixsecurity21/presentation/dalskov>
12. Damgård, I., Fitzi, M., Kiltz, E., Nielsen, J.B., Toft, T.: Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation. In: *Theory of Cryptography Conference*. pp. 285–304. Springer (2006). https://doi.org/10.1007/11681878_15
13. Damgård, I., Nielsen, J.B.: Scalable and unconditionally secure multiparty computation. In: *Annual International Cryptology Conference*. pp. 572–590. Springer (2007). https://doi.org/10.1007/978-3-540-74143-5_32
14. Damgård, I., Pastro, V., Smart, N., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: *Annual Cryptology Conference*. pp. 643–662. Springer (2012). https://doi.org/10.1007/978-3-642-32009-5_38
15. Escudero, D., Ghosh, S., Keller, M., Rachuri, R., Scholl, P.: Improved primitives for mpc over mixed arithmetic-binary circuits. In: *Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part II* 40. pp. 823–852. Springer (2020). https://doi.org/10.1007/978-3-030-56880-1_29
16. Eskandarian, S., Boneh, D.: Clarion: Anonymous communication from multiparty shuffling protocols. *Cryptology ePrint Archive* (2021). <https://doi.org/10.14722/ndss.2022.24141>
17. Gordon, S.D., Le, P.H., McVicker, D.: Linear communication in malicious majority mpc. p. 2173–2187. *CCS ’23, Association for Computing Machinery, New York, NY, USA* (2023). <https://doi.org/10.1145/3576915.3623162>, <https://doi.org/10.1145/3576915.3623162>
18. Goyal, V., Song, Y., Zhu, C.: Guaranteed Output Delivery Comes Free in Honest Majority MPC. In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology – CRYPTO 2020*. pp. 618–646. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-56880-1_22
19. Groth, J.: A verifiable secret shuffle of homomorphic encryptions. *Journal of Cryptology* **23**, 546–579 (2010). https://doi.org/10.1007/3-540-36288-6_11
20. Hamada, K., Ikarashi, D., Chida, K., Takahashi, K.: Oblivious radix sort: An efficient sorting algorithm for practical secure multi-party computation. *Cryptology ePrint Archive* (2014)
21. Hamada, K., Kikuchi, R., Ikarashi, D., Chida, K., Takahashi, K.: Practically efficient multi-party sorting protocols from comparison sort algorithms. In: *Information Security and Cryptology–ICISC 2012: 15th International Conference, Seoul, Korea, November 28–30, 2012, Revised Selected Papers* 15. pp. 202–216. Springer (2013). https://doi.org/10.1007/978-3-642-37682-5_15
22. Han, F., Lan, X., Liu, W., Zhang, L., Ren, H., Qu, L., Hong, Y.: Concretely efficient correlated oblivious permutation. *Cryptology ePrint Archive* (2025)

23. Hohenberger, S., Rothblum, G.N., Shelat, A., Vaikuntanathan, V.: Securely obfuscating re-encryption. In: Theory of Cryptography Conference. pp. 233–252. Springer (2007). https://doi.org/10.1007/978-3-540-70936-7_13
24. Kanagavelu, R., Li, Z., Samsudin, J., Yang, Y., Yang, F., Mong Goh, R.S., Cheah, M., Wiwatphonthana, P., Akkarajitsakul, K., Wang, S.: Two-phase multi-party computation enabled privacy-preserving federated learning. In: 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID). pp. 410–419 (2020). <https://doi.org/10.1109/CCGrid49817.2020.00-52>
25. Keller, M.: MP-SPDZ: A versatile framework for multi-party computation. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (2020). <https://doi.org/10.1145/3372297.3417872>
26. Keller, M., Orsini, E., Scholl, P.: Mascot: faster malicious arithmetic secure computation with oblivious transfer. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 830–842 (2016). <https://doi.org/10.1145/2976749.2978357>
27. Keller, M., Pastro, V., Rotaru, D.: Overdrive: Making spdz great again. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 158–189. Springer (2018)
28. Keller, M., Scholl, P.: Efficient, oblivious data structures for MPC. In: Sarkar, P., Iwata, T. (eds.) Advances in Cryptology – ASIACRYPT 2014. pp. 506–525. Springer
29. Laud, P.: Linear-time oblivious permutations for spdz. In: Cryptology and Network Security: 20th International Conference, CANS 2021, Vienna, Austria, December 13–15, 2021, Proceedings 20. pp. 245–252. Springer (2021). https://doi.org/10.1007/978-3-030-92548-2_13
30. Laud, P., Pettai, M.: Secure multiparty sorting protocols with covert privacy. In: Secure IT Systems: 21st Nordic Conference, NordSec 2016, Oulu, Finland, November 2–4, 2016. Proceedings 21. pp. 216–231. Springer (2016). https://doi.org/10.1007/978-3-319-47560-8_14
31. Laur, S., Willemson, J., Zhang, B.: Round-efficient oblivious database manipulation. In: Information Security: 14th International Conference, ISC 2011, Xi’an, China, October 26–29, 2011. Proceedings 14. pp. 262–277. Springer (2011). https://doi.org/10.1007/978-3-642-24861-0_18
32. Mohassel, P., Rindal, P.: Aby3: A mixed protocol framework for machine learning. In: Proceedings of the 2018 ACM SIGSAC conference on computer and communications security. pp. 35–52 (2018)
33. Mugunthan, V., Polychroniadou, A., Byrd, D., Balch, T.H.: Smpai: Secure multi-party computation for federated learning. In: Proceedings of the NeurIPS 2019 Workshop on Robust AI in Financial Services. vol. 21. MIT Press Cambridge, MA, USA (2019)
34. Peceny, S., Raghuraman, S., Rindal, P., Shah, H.: Efficient permutation correlations and batched random access for two-party computation. In: IACR International Conference on Public-Key Cryptography. pp. 76–109. Springer (2025)
35. Song, X., Yin, D., Bai, J., Dong, C., Chang, E.C.: Secret-shared shuffle with malicious security. Network and Distributed System Security (NDSS) Symposium (2024). <https://doi.org/10.14722/ndss.2024.24021>
36. Soththiwat, E., Zhen, L., Li, Z., Zhang, C.: Partially encrypted multi-party computation for federated learning. In: 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid). pp. 828–835. IEEE (2021). <https://doi.org/10.1109/CCGrid51090.2021.00101>

37. Volgushev, N., Schwarzkopf, M., Getchell, B., Varia, M., Lapets, A., Bestavros, A.: Conclave: secure multi-party computation on big data. EuroSys '19, Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3302424.3303982>

A Case Studies

In this section, we present several concrete protocols generated by instantiating our construction with existing MPC permutation protocols from [35], [28] and [31], respectively. The first construction applies only to additive secret sharing, and is highly efficient, due to utilizing possibly currently most efficient MPC permutation protocol. The second construction works for arbitrary MPC framework supporting MPC multiplication and has a relatively simple implementation. The third construction may be adapted to any threshold secret sharing scheme, but requires ZK proofs in its offline phase to achieve malicious security.

Despite all these differences, all three constructions are actively secure and have linear online complexity, due to our malicious-secure shuffle correlation.

A.1 Shuffle Protocol for Additive Secret Sharing

The construction of Song et al. [35] is based on SPDZ framework [14], where a variable stored at MPC is additively secret-shared among all parties, along with an additional MAC. That is, the statement “a variable a is secret-shared as $\langle a \rangle$ ” means that each party P_i holds locally a tuple $(a_i, \gamma_i(a))$ such that

$$\sum_{i=1}^n a_i = a, \quad \sum_{i=1}^n \gamma_i(a) = \alpha a,$$

where α is a secret global key that is used to guarantee the integrity of opening. Song et al. propose a two-party (sender and receiver) permutation protocol, which permutes a secret-shared array with permutation specified by one of the party (i.e. the receiver).

Basically, this two-party permutation protocol is built from repeating basic CGP protocol [7] for B times with additional correctness checks, while B is a parameter required for applying cut-and-choose technique. If the receiver has chosen permutation $\pi_1, \dots, \pi_B$ respectively in these sessions, the overall effect is that the array is permuted by a single permutation π :

$$\pi = \pi_B \circ \pi_{B-1} \circ \dots \circ \pi_1.$$

This two-party permutation protocol is then extended to the case of n -party by letting a fixed party P_i act as the receiver in all $(n-1) \times B$ sessions, which can be interpreted as permuting each party’s shares with the same set of B permutations. Of course, additional steps are taken in [35] to prevent P_i from acting inconsistently in sessions with different parties, e.g. choosing different B permutations when the sender is different. The protocol then checks if each session of the CGP protocol finishes correctly, to guarantee that no misbehavior happens so far.

To conclude, the work of Song et al. [35] provides us with exactly the functionality

$$(\langle \pi(\mathbf{x}_1) \rangle, \dots, \langle \pi(\mathbf{x}_t) \rangle) \leftarrow \Pi_{\text{perm}}(P_i : \pi, \langle \mathbf{x}_1 \rangle, \dots, \langle \mathbf{x}_t \rangle),$$

with an overhead of $O(Bnmt \log m)$ communication and computation complexity. By instantiating with the SPDZ framework of [17] and fitting above permutation protocol into Algorithm 5, we obtain a shuffle protocol with $O(Bn^2m \log m)$ offline communication and computation complexity, whose online complexity is linear.

One intricate detail regarding implementation is that at the end of the shuffle protocol, it must be guaranteed that the array is correctly shuffled. This arises in applications where the shuffle is followed by a data opening, such as the Clarion anonymous communication system designed by [16], which opens all values after the shuffle. However, SPDZ itself does not guarantee safe opening, i.e. before the MAC check passes, the correctness of all opened values is not guaranteed. This means that Π_{open} does not guarantee a safe open (which we have assumed throughout), which means that all values during the shuffle protocol, including those used in verification, could be wrong, and the correctness is not guaranteed even if the protocol doesn't abort. Luckily, SPDZ framework supports immediate MAC check, which checks if all previously opened values are correct, with only $O(n)$ communication and computation complexity. Therefore, in practice, the shuffle protocol must be followed by such an immediate MAC check, which in turn makes all previous Π_{open} safe. We remark that this is also the approach followed by Song et al., as their shuffle protocol ends with precisely the MAC check of SPDZ.

We note that, this results in the first maliciously secure shuffle protocol for additive secret sharing with linear online complexities.

A.2 Shuffle Protocol for Shamir Secret Sharing

The construction of permutation protocol in [28] works for arbitrary MPC framework supporting MPC multiplication. Assuming MPC multiplication can be done within $O(n)$ communication and computation, its implementation of Π_{perm} requires $O(nm \log m)$ communication and computation in total.

The permutation protocol developed in [28] utilizes a specific permutation network, which is a switching network of size $O(m \log m)$ and of depth $O(\log m)$. Such a network consists of $O(\log m)$ layers of switches, each switch randomly swapping two entries of the vector depending on an extra control bit. As a permutation network is capable of representing any permutation by $O(m \log m)$ switches, this allows parties to represent a permutation by $O(m \log m)$ bits.

Now suppose P_i wants to permute $\langle \mathbf{x} \rangle$ by permutation π . P_i first represents π by permutation network, which results in $O(m \log m)$ control bits, each corresponding to the control bit of one switch. P_i then shares these bits to parties, who then check together whether the shared values are bits. This check can be realized by MPC multiplication, by noting that $\langle b \rangle$ is a bit if and only if $b \cdot (1 - b)$ is zero. By following the permutation network and using MPC multiplication to simulate each switch, the parties are now able to permute any m -long vector by π .

Such an implementation matches exactly our requirements for Π_{perm} . By instantiating Π_{perm} in Algorithm 5 with such implementation, we immediately

obtain a shuffle protocol for arbitrary MPC framework supporting MPC multiplication. The offline phase consists of $O(nm \log m)$ many multiplications, which results in $O(n^2m \log m)$ communication, computation and $O(\log m)$ rounds, assuming the communication and computation overhead of each multiplication are both linear. By instantiating with the Shamir secret sharing scheme in [18], we obtain a concrete construction that meets above complexities. The above implementation can also be instantiated with SPDZ framework of [17], with $O(n^2m \log m)$ offline complexities that is asymptotically better than instantiating with the permutation protocol of [35]. Nevertheless, since the primitives used in [35] might be cheaper than MPC multiplication, which construction is more concretely efficient may depend on concrete implementation.

We note that, this results in the first shuffle protocols for Shamir secret sharing with linear online complexities, for both semi-honest and malicious security.

A.3 Shuffle Protocol instantiated with [29]

The construction of Laur et al. [31] works for threshold secret sharing, with possibly malicious adversaries. By its design, each party belongs to several groups, and each group of parties will agree on a common permutation and permute the array once. The size of each group is large enough, so that it is possible to reconstruct all secret with the shares of the group member, which allows the group to permute the array trivially by each member permuting the shares locally and re-distributing their shares. For $(n/2)$ -threshold secret sharing, there will be asymptotically $O(2^n/\sqrt{n})$ many groups, and by design at least one of the groups consists of only honest parties. Hence, since this all-honest group permutes the array with a uniform random permutation known only to its group members, the overall permutation is hidden from the adversary. The advantage of this construction is that, when the number of parties are small and all parties are semi-honest, it does not utilize any public key primitives and is extremely fast. Also, most of the other MPC shuffle protocols works only with additive secret sharing (e.g. [7] [29] [16] [35]), while the construction of [31] works with threshold secret sharing, e.g. Shamir secret sharing.

It is possible to extend our construction, so that the shuffle protocol may be instantiated by the protocol of Laur et al. [31]. For clarity, suppose we are working in semi-honest case. Suppose we have t such groups

$$\mathcal{G} = \{G_1, G_2, \dots, G_t\},$$

by the design of [31], with $G_i \subseteq \{P_i\}_{i \in [n]}$. All that needs to be done is to replace the Π_{perm} functionality by

$$\llbracket \mathbf{x} \rrbracket \leftarrow \Pi_{\text{perm}}(G : \pi, \llbracket \mathbf{x} \rrbracket),$$

where $G \in \mathcal{G}$, i.e. π is known only to members of G . The rest of the protocol proceeds naturally by treating each G_i as an individual participant, i.e. generating and permuting random vectors with each permutation π_i known only to members of G_i , opening $\mathbf{z}_i$ to only members of G_i , etc. The online phase hence consists

of $O(2^n/\sqrt{n})$ rounds and has $O(2^n m/\sqrt{n})$ communication complexity. Note that instead of repeating this same process on all members of G_i , each group G_i can elect a party in the group as its agent, who will permute the masked vector $\mathbf{y}_{i-1}$, add the vector $\mathbf{z}_i$ and send $\mathbf{y}_i$ to agent of the next group G_{i+1} .

Further, to reduce the online round complexity to $O(n)$ and online communication to $O(nm)$, we can sort the groups in a particular order, such that each P_i is the agent of only a consecutive sequence of groups. For example, if

$$\mathcal{G} = (\{P_1, P_2\}, \{P_2, P_3\}, \{P_1, P_3\}),$$

we can sort it as

$$\mathcal{G} = (\{P_1, P_2\}, \{P_1, P_3\}, \{P_2, P_3\}).$$

Now, by letting P_1 be the agent of G_1 and G_2 , P_1 will be responsible for sending both $\mathbf{y}_1$ and $\mathbf{y}_2$ in the online phase, and sending $\mathbf{y}_2$ to the agent of G_3 . It is clear that P_1 can carry out the computation locally, and sends only $\mathbf{y}_2$ to the agent of G_3 , which consumes only 1 round instead of 2. By letting a consecutive sequence to have a common agent, this results in an $O(n)$ -round online phase with $O(nm)$ communication.

The online computation can also be made $O(nm)$. Since agent P_i knows all the permutations and masks it needs in online phase, it can carry out the data independent part of the computation in offline phase. To be more specific, suppose it needs to compute

$$\begin{aligned} \mathbf{y}_1 &= \pi_1(\mathbf{x} + \mathbf{z}_1), \\ \mathbf{y}_2 &= \pi_2(\mathbf{y}_1 + \mathbf{z}_2), \\ \mathbf{y}_3 &= \pi_3(\mathbf{y}_2 + \mathbf{z}_3), \\ &\vdots \\ \mathbf{y}_k &= \pi_k(\mathbf{y}_{k-1} + \mathbf{z}_k). \end{aligned}$$

This is simplified to compute

$$\begin{aligned} \mathbf{y}_k &= \pi_k \circ \pi_{k-1} \circ \cdots \circ \pi_1(\mathbf{y}_1) \\ &\quad + \pi_k \circ \pi_{k-1} \circ \cdots \circ \pi_1(\mathbf{z}_1) \\ &\quad + \pi_k \circ \pi_{k-1} \circ \cdots \circ \pi_2(\mathbf{z}_2) \\ &\quad + \pi_k \circ \pi_{k-1} \circ \cdots \circ \pi_3(\mathbf{z}_3) \\ &\quad \vdots \\ &\quad + \pi_k(\mathbf{z}_k). \end{aligned}$$

It is clear that the permutations of $\mathbf{z}_i$ can be computed and summed up in offline phase. Also, the party can first compute the permutation $\pi_k \circ \cdots \circ \pi_1$, and hence in the online phase it needs to only perform one permutation for $\mathbf{y}_1$ and add it with the pre-processed sum.

This can be also extended to malicious security, with the same technique used earlier in Section 6. Note that we only need to check if the final result of agent P_i

is correct, since all other computations are done internal P_i . Hence, the overall online computation complexity remains $O(nm)$. Note that as the construction of [31] utilizes ZK proof for malicious security, it is likely that the construction instantiated with [35] or [28] would be more efficient in practice. Nevertheless, if one wishes to build their MPC application with Shamir secret sharing or replicated secret sharing (instead of additive secret sharing), the construction of [35] will not be available, and instantiating shuffle protocol with the construction of [31] might be a good choice.

B Security Proof for Semi-honest Secure Shuffle

Definition 5 (Security for Semi-Honest Shuffle). *Suppose that adversary $\mathcal{A}$ can corrupt up to $n - 1$ parties. The parties are to execute protocol $\text{Shuffle}_{\text{off}}$ followed by a protocol $\text{Shuffle}_{\text{on}}$ with the correlation just generated. After the protocol $\text{Shuffle}_{\text{off}}$ completes, $\mathcal{A}$ is to choose a vector $\mathbf{x}$, and input it with Π_{input} . At the end of the protocol $\text{Shuffle}_{\text{on}}$, $\mathcal{A}$ obtains an overall view, denoted by $\text{view}_{\mathcal{A}}$, which contains all values observed the the corrupted parties during the execution.*

Let $T \subsetneq [n]$ denote the set of corrupted parties, and $\bar{T}$ the set of honest parties. The shuffle protocol is said to be secure against semi-honest adversaries, if for any possible permutations $(\pi'_i)_{i \in \bar{T}}$,

$$\Pr[(\pi_i)_{i \in \bar{T}} = (\pi'_i)_{i \in \bar{T}} \mid \text{view}_{\mathcal{A}}] = \frac{1}{(m!)^{|\bar{T}|}}.$$

That is, the tuple $(\pi_i)_{i \in \bar{T}}$ of permutations chosen by the honest parties $P_i \in \bar{T}$ is uniform over all possible permutations, even when conditioned on the view of the adversary. This can be expressed equivalently via mutual information, by stating

$$I((\pi_i)_{i \in \bar{T}}; \text{view}_{\mathcal{A}}) = 0,$$

i.e. the permutations of honest parties are independent of the adversary's view.

Theorem 2. *The semi-honest two-phase shuffle protocol $\text{Shuffle}_{\text{off}}$ and $\text{Shuffle}_{\text{on}}$ is secure against semi-honest adversaries, in the sense of Definition 5. That is, the view of any adversary corrupting up to $n - 1$ parties is independent of the permutations of honest parties, i.e.*

$$I((\pi_i)_{i \in \bar{T}}; \text{view}_{\mathcal{A}}) = 0.$$

Proof. The core idea of the proof is that, fixing the adversary's view and $\{\pi_i\}_{i \in \bar{T}}$ of permutations chosen by the honest parties, for any arbitrary permutations $\{\pi'_i\}_{i \in \bar{T}}$, there is exactly one assignment to $(\mathbf{r}_i)_{i \in [n]}$ and $(\mathbf{z}_i)_{i \in \bar{T}}$, such that the adversary's view is not altered, yet the output of the protocol is $\mathbf{x}$ shuffled with $\{\pi'_i\}_{i \in \bar{T}}$ instead of $\{\pi_i\}_{i \in \bar{T}}$.

For a clear demonstration, assume that the adversary corrupts exactly $n - 1$ parties, leaving P_k the only honest party. The adversary's view consists of

$$\text{view}_{\mathcal{A}} = \begin{pmatrix} \mathbf{x} \\ \pi_1 \ \pi_2 \ \cdots \ \pi_{k-1} & \pi_{k+1} \ \cdots \ \pi_n \\ \mathbf{z}_1 \ \mathbf{z}_2 \ \cdots \ \mathbf{z}_{k-1} & \mathbf{z}_{k+1} \ \cdots \ \mathbf{z}_n \\ \mathbf{y}_1 \ \mathbf{y}_2 \ \cdots \ \mathbf{y}_{k-1} \ \mathbf{y}_k & \mathbf{y}_{k+1} \ \cdots \ \mathbf{y}_n \end{pmatrix}.$$

Note that the view contains $\mathbf{y}_k$, since it's the message from honest P_k to corrupted P_{k+1} .

Let's suppose honest party P_k has originally chosen π_k , but now we want the result to be as if it has chosen π'_k . we claim that there exists a unique assignment to $\{\mathbf{z}_i, \mathbf{r}_i, \pi_i\}_{i \in [n]}$, denoted by $(\mathbf{z}'_i)_{i \in [n]}$, $(\mathbf{r}'_i)_{i \in [n]}$ and $(\pi'_i)_{i \in [n]}$, such that

1. For each $i \neq k$, $\pi'_i = \pi_i$.
2. For each $i \neq k$, $\mathbf{z}_i = \pi_{i-1}(\mathbf{r}_{i-1}) - \mathbf{r}_i = \pi'_{i-1}(\mathbf{r}'_{i-1}) - \mathbf{r}'_i = \mathbf{z}'_i$.
3. For P_k , $\mathbf{z}'_k = \pi'_{k-1}(\mathbf{r}'_{k-1}) - \mathbf{r}'_k$, which may differ from the original $\mathbf{z}_k$.
4. The output will be

$$\llbracket \bar{\pi}'_n \rrbracket := \llbracket \pi'_n \circ \pi'_{n-1} \circ \cdots \circ \pi'_1(\mathbf{x}) \rrbracket.$$

Note that the above statements imply that the view of corrupted parties remains unchanged by replacing π_k with arbitrary π'_k , and even remains consistent with its newly given semantics (i.e. shuffling with π'_i). Hence, since π'_k is an arbitrary permutation, this indicates that any permutation π'_k is equally possible in the view of adversary.

This exact assignment can be solved from the constraints posed by the view. Suppose first $k = 1$, note that

$$\mathbf{z}_1 = \mathbf{x} - \mathbf{r}_1, \ \mathbf{y}_1 = \pi_1(\mathbf{z}_1).$$

Since $\mathbf{y}_1$ appears in the view, it demands $\mathbf{z}'_1 = \pi'^{-1}_1(\mathbf{y}_1)$. Luckily, $\mathbf{z}_1$ and $\mathbf{z}'_1$ does not appear in the view, hence they can be different. This uniquely determines $\mathbf{r}'_1 = \mathbf{x} - \pi'^{-1}_1(\mathbf{y}_1)$. Also, observe that

$$\mathbf{y}_1 = \pi_1(\mathbf{x}) - \pi_1(\mathbf{r}_1) = \pi'_1(\mathbf{x}) - \pi'_1(\mathbf{r}'_1) = \mathbf{y}'_1,$$

which coincides with the requirement of $\mathbf{y}_1 = \mathbf{y}'_1$.

Now that it is demanded that $\mathbf{z}_2 = \mathbf{z}'_2 = \pi'_1(\mathbf{r}'_1) - \mathbf{r}'_2$, this gives exactly one solution to $\mathbf{r}'_2 = \pi'_1(\mathbf{r}'_1) - \mathbf{z}_2$. Since $\pi_2 = \pi'_2$, $\mathbf{y}'_1 = \mathbf{y}_1$ and $\mathbf{z}'_2 = \mathbf{z}_2$, we have $\mathbf{y}'_2 = \mathbf{y}_2$ naturally. Furthermore, it can be checked that

$$\mathbf{y}'_2 = \mathbf{y}_2 = \pi_2(\mathbf{y}_1 + \mathbf{z}_2) = \bar{\pi}'_2(\mathbf{x}) - \pi'_2(\mathbf{r}'_2),$$

which follows trivially from the fact that

$$\mathbf{y}_1 = \mathbf{y}'_1 = \pi'_1(\mathbf{x}) - \pi'_1(\mathbf{r}'_1),$$

and

$$\mathbf{z}_2 = \mathbf{z}'_2 = \pi'_1(\mathbf{r}'_1) - \mathbf{r}'_2,$$

and $\pi_2 = \pi'_2$.

Now following this path, we can assign a new value to each $\mathbf{r}_i$ as $\mathbf{r}'_i$ for $i \geq 2$, such that $\mathbf{z}'_i = \mathbf{z}_i$, yet the “explanation” of $\mathbf{y}_i$ is replaced by

$$\mathbf{y}_i = \mathbf{y}'_i = \bar{\pi}'_i(\mathbf{x}) - \pi'_i(\mathbf{r}'_i).$$

And at the end, party P_n obtains

$$\mathbf{y}_n = \mathbf{y}'_n = \bar{\pi}'_n(\mathbf{x}) - \pi'_n(\mathbf{r}'_n).$$

After broadcasting and adding it with $\llbracket \pi'_n(\mathbf{r}'_n) \rrbracket$, the result is indeed $\bar{\pi}'_n(\mathbf{x})$.

Hence, if P_1 is honest, by seeing only the values in the view, π_1 can be any permutation π'_1 . And the probabilities of π_1 being any π'_1 are equal, since there is exactly one set of assignment to $\mathbf{r}_i$ that supports the claim $\pi_1 = \pi'_1$. As the adversary does not know any $\mathbf{r}_i$ or $\pi_i(\mathbf{r}_i)$, it cannot prefer any value of π_1 over another. Thus, the choice of π_1 is independent of the view of $\mathcal{A}$.

The argument for $k \geq 2$ proceeds similarly. For $k \geq 2$, all $\mathbf{r}_i$ for $i < k$ are fully determined by the adversary’s view and hence cannot be altered. However, since the adversary cannot observe $\mathbf{z}_k$, we retain the flexibility to adjust $\mathbf{r}_k$ and π_k arbitrarily.

The general case where $\mathcal{A}$ corrupts an arbitrary subset $T \subsetneq [n]$ follows by analogous reasoning, which we omit here for brevity. $\square$

C Security Proof for Maliciously Secure Shuffle

C.1 Roadmap and Ideal Functionality

Before we dive into a formal security proof, let’s give some intuitions on why the offline and online protocol are secure.

First recall that the offline phase $\text{Shuffle}_{\text{off}}$ of the shuffle protocol essentially performs only randomness generation, multiplications, calls to Π_{perm} , and openings. Since all these operations are supported by $\mathcal{F}_{\text{MPC}}$, the security of offline phase is immediate.

In the online phase $\text{Shuffle}_{\text{on}}$, the potential security risk stems from message $\mathbf{y}_i$. Since $\mathbf{z}_i$ and π_i is by design only known to party P_i , honest parties could not tell if what P_i sends is indeed $\mathbf{y}_i = \pi_i(\mathbf{z}_i + \mathbf{y}_{i-1})$. However, note that in our construction, the parties will be able to check whether the correlation implanted in $\mathbf{y}_i$ is sound by PartialVerify . Intuitively, as β and $\mathbf{r}'_i$ are both never revealed, P_i cannot forge a fake $\mathbf{y}'_i$ and still pass the test. This guarantees that only correct $\mathbf{y}_i$ can pass the test, which roughly means that the adversary must behave honestly.

Hence, in this section, we prove first that adversary cannot learn the shuffle authentication key β before entering any PartialCheck , even if it deviates from the protocol. This is, of course, of vital importance. Then, we will prove that due to having no knowledge to β , the adversary cannot forge $\mathbf{y}_i$ and pass the test with non-negligible probability, nor can it forge w_1 and w_2 broadcast in PartialVerify and pass the test with non-negligible probability. This means that if the protocol

does not abort, w.h.p. all messages sent to honest parties are correctly computed according to the protocol, which roughly makes the adversary semi-honest. Finally, we construct a simulator for the entire process, which either aborts with same probability at the same step in a real execution, or proceeds and outputs a view indistinguishable from the view in real execution, which finally concludes the UC security.

Below, we assume the adversary $\mathcal{A}$ has corrupted all parties in $T \subsetneq [n]$, leaving $\bar{T}$ the set of honest parties.

The formal description of the functionality of the protocol is as follows. The security is defined by the indistinguishability between the execution of protocol in real world with an execution of the protocol in ideal world, with the help of ideal functionality $\mathcal{F}_{\text{shuffle}}$.

Definition 6. (*Ideal Functionality of $\text{Shuffle}_{\text{off}}$*) Denote by $\mathcal{F}_{\text{off}}$ an ideal functionality, which is $\mathcal{F}_{\text{MPC}}$ equipped with an additional (ideal) command Π_{off} . This command takes as input a public length m and each m -permutation π_i from P_i , aborts if π_i is not a valid m -permutation or parties do not agree on m , and otherwise generates a shuffle correlation. That is,

$$\text{cor} = \begin{pmatrix} \langle \beta \rangle & \langle \mathbf{r} \rangle & \langle \beta \mathbf{r} \rangle & \langle \pi_1^{-1}(\mathbf{r}'_1) \rangle & \langle \mathbf{s} \rangle \\ \pi_1 & \pi_2 & \pi_3 & \pi_4 & \pi_5 & \cdots & \pi_n \\ \langle \mathbf{r}'_1 \rangle & \langle \mathbf{r}'_2 \rangle & \langle \mathbf{r}'_3 \rangle & \langle \mathbf{r}'_4 \rangle & \langle \mathbf{r}'_5 \rangle & \cdots & \langle \mathbf{r}'_n \rangle \\ & \langle c_2 \rangle & \langle c_3 \rangle & \langle c_4 \rangle & \langle c_5 \rangle & \cdots & \langle c_n \rangle \\ & \langle d_2 \rangle & \langle d_3 \rangle & \langle d_4 \rangle & \langle d_5 \rangle & \cdots & \langle d_n \rangle \\ & \mathbf{z}_2 & \mathbf{z}_3 & \mathbf{z}_4 & \mathbf{z}_5 & \cdots & \mathbf{z}_n \end{pmatrix},$$

which satisfies Definition 4.

Remark. As mentioned earlier, under $\mathcal{F}_{\text{MPC}}$ -hybrid model, we can prove that π_i is protected even upon abort. Hence we here slightly abuse the notation, and treat π_i as secret input from P_i , which must not be leaked even if the protocol aborts.

Definition 7. (*Ideal Functionality of Shuffle*) Denote by $\mathcal{F}_{\text{shuffle}}$ an ideal functionality, which is the ideal functionality $\mathcal{F}_{\text{MPC}}$ equipped with an additional command Π_{shuffle} . This command takes as input $\langle \mathbf{x} \rangle$, and stores $\langle \pi(\mathbf{x}) \rangle$ in the ideal functionality, where π is uniformly randomly drawn by $\mathcal{F}_{\text{shuffle}}$.

We begin with three lemmas. The first states that the offline phase is secure, in the sense that the adversary cannot learn information more than $\mathbf{z}_i$ that is revealed to it. The second and third describe the security of protocol Verify and PartialVerify, respectively.

Lemma 1 (Security of Offline Phase). *The protocol $\text{Shuffle}_{\text{off}}$ is UC secure in the $\mathcal{F}_{\text{MPC}}$ -hybrid model. Further, the view of adversary (besides m) in offline phase is equivalent to*

$$\text{view}_{\mathcal{A}} = \{\pi_i, \mathbf{z}_i\}_{i \in T}.$$

Note that since $\mathbf{z}_1$ is not included in the offline phase, it should be understood as a null value if P_1 is corrupted.

Proof. The protocol $\text{Shuffle}_{\text{off}}$ is almost secure by definition. Nevertheless, as a warming up, we prove the security by constructing the simulator.

First, we note that the protocol $\text{Shuffle}_{\text{off}}$ correctly implements ideal functionality $\mathcal{F}_{\text{off}}$ by outputting a correlation that satisfies Definition 4. This can be verified directly by first noting that the constraints are satisfied. Moreover, note that since in the protocol,

$$\begin{aligned}\langle \mathbf{z}_i^1 \rangle &\leftarrow \langle \pi_{i-1}(\mathbf{r}_{i-1}) \rangle - \langle \mathbf{r}_i \rangle, \\ \langle \mathbf{z}_i^2 \rangle &\leftarrow \langle \pi_{i-1}(\beta \mathbf{r}_{i-1}) \rangle - \langle \beta \mathbf{r}_i \rangle + \langle \pi_{i-1}(\mathbf{r}'_{i-1}) \rangle - \langle \mathbf{r}'_i \rangle.\end{aligned}$$

This means that after fixing $\pi_i, \beta, \mathbf{r}'_i, \mathbf{r}_1$ and $\mathbf{r}_n$, there is a bijection between set

$$\{(\mathbf{r}_i)_{2 \leq i \leq n-1} : \mathbf{r}_i \in \mathbb{F}^m\} \text{ and } \{(\mathbf{z}_i)_{2 \leq i \leq n} : \text{constraint is satisfied}\}.$$

Since the protocol generates $\mathbf{r}_i$ for $i \in \{2, 3, \dots, n-1\}$ uniformly, this means $\mathbf{z}_i$ will be uniform under the constraint, as required by definition.

Consider a simulator $\mathcal{S}$ that acts as follows. At the start of the game, $\mathcal{E}$ first sends to $\mathcal{F}_{\text{off}}$ the input of honest parties for command Π_{off} , which is π_i for P_i . $\mathcal{S}$ simply follows the protocol, since $\mathcal{A}$ will obtain no output by calling to ideal $\mathcal{F}_{\text{MPC}}$ in the real world. In the step of calling $\Pi_{\text{perm}}(P_i : \pi_i, \dots)$ for corrupted party P_i , since in the real world execution $\mathcal{A}$ will need to send π_i to Π_{perm} , $\mathcal{S}$ will receive (purported) π_i from $\mathcal{E}$. Hence, $\mathcal{S}$ can send π_i to $\mathcal{F}_{\text{off}}$, as part of the input required by command Π_{off} .

When all inputs are gathered, $\mathcal{F}_{\text{off}}$ will send $\mathbf{z}_i$ in the shuffle correlation to $\mathcal{S}$, for each corrupted P_i with $i \geq 2$. $\mathcal{S}$ is then able to simulate the opening of $\mathbf{z}_i$ for corrupted party P_i , as if P_i has received $\mathbf{z}_i$ in the real world.

Note also that during the process, whenever $\mathcal{E}$ demands $\mathcal{S}$ to abort, $\mathcal{S}$ simply aborts as $\mathcal{A}$ does. If the protocol does not eventually abort, $\mathcal{S}$ sends “continue” to $\mathcal{F}_{\text{off}}$, who then reveals its final state (i.e. the entire shuffle correlation cor) to $\mathcal{E}$. This shuffle correlation is of course consistent with the view of adversary $\mathcal{A}$, as the $\mathbf{z}_i$ and π_i are the same. Hence, the above process is perfect, i.e. is identical with the execution in the real world.

Additionally, it is also clear from above simulation that the adversary’s view is $\mathbf{z}_i$ opened to it during the protocol, plus the π_i chosen by itself. $\square$

Lemma 2. *The protocol $\text{Verify}(\mathbf{a}, \mathbf{b}, \langle \beta \rangle, \langle \mathbf{r} \rangle)$ is secure, in that if $\beta \mathbf{a} \neq \mathbf{b} + \mathbf{r}$, the test fails immediately and all parties abort.*

Proof. Suppose $\beta \mathbf{a} \neq \mathbf{b} + \mathbf{r}$. In protocol, parties compute and open

$$\mathbf{d} \leftarrow \mathbf{a} \langle \beta \rangle - \mathbf{b} - \langle \mathbf{r} \rangle.$$

Since the algorithmic operations are done via ideal functionality $\mathcal{F}_{\text{MPC}}$, the adversary cannot bias the value of opened $\langle \mathbf{d} \rangle$. Hence, if $\mathbf{d} \neq \mathbf{0}$, the parties will abort. $\square$

Lemma 3. *The protocol $\text{PartialVerify}(P_{i+1}, \mathbf{y}_i^1, \mathbf{y}_i^2, \langle \beta \rangle, \langle c_{i+1} \rangle, \langle d_{i+1} \rangle)$ is secure, in the sense that if honest P_{i+1} does not receive valid $\mathbf{y}_i$ such that $\beta \mathbf{y}_i^1 = \mathbf{y}_i^2 + \pi_i(\mathbf{r}'_i)$, it aborts with probability*

$$p \geq 1 - \frac{m-1}{2^\kappa}.$$

Remark. We remark that receiving “valid” $\mathbf{y}_i$ such that $\beta \mathbf{y}_i^1 = \mathbf{y}_i^2 + \pi_i(\mathbf{r}'_i)$ is slightly different from receiving “correct” $\mathbf{y}_i$ specified by the protocol, since the adversary might (somehow) forge $\mathbf{y}'_i \neq \mathbf{y}_i$ that satisfies this correlation.

This lemma claims only that upon receiving $\mathbf{y}_i$ that does not satisfy the correlation, the parties abort w.h.p. Later we will prove that the adversary cannot forge $\mathbf{y}'_i \neq \mathbf{y}_i$ that satisfies this correlation. This makes the statement “receiving valid $\mathbf{y}_i$ ” equivalent to “receiving correct $\mathbf{y}_i$ ” (except with negligible probability).

Proof. Suppose P_{i+1} is honest, and $\beta \mathbf{y}_i^1 \neq \mathbf{y}_i^2 + \pi_i(\mathbf{r}'_i)$. By Lemma 1, the offline phase of the shuffle is secure. Thus, c_{i+1} is uniformly random, and

$$d_{i+1} = \sum_{j=1}^m c_{i+1}^{j-1} \pi_i(\mathbf{r}'_i)(j).$$

Consider the polynomial

$$f(X) = \sum_{j=1}^m (\beta \cdot \mathbf{y}_i^1(j) - \mathbf{y}_i^2(j) - \pi_i(\mathbf{r}'_i)(j)) \cdot X^{j-1}.$$

As $\beta \mathbf{y}_i^1 \neq \mathbf{y}_i^2 + \pi_i(\mathbf{r}'_i)$, f is a non-zero polynomial of degree at most $m - 1$. Note that the protocol PartialVerify opens exactly $f(c_{i+1})$ to all parties. As c_{i+1} appears only in $\langle c_{i+1} \rangle$ and $\langle d_{i+1} \rangle$, which are never opened before the protocol PartialVerify, c_{i+1} appears uniformly random to the adversary. Thus, as f has at most $m - 1$ roots in $\mathbb{F}$, c_{i+1} happens to be one of them with probability

$$p_{\text{pass}} = \Pr(f(c_{i+1}) = 0 \mid c_{i+1} \leftarrow \mathbb{F}) \leq \frac{m-1}{|\mathbb{F}|}.$$

Thus,

$$p_{\text{abort}} = 1 - p_{\text{pass}} \geq 1 - \frac{m-1}{|\mathbb{F}|} \geq 1 - \frac{m-1}{2^\kappa}.$$

□

C.2 Ignorance of Shuffle Authentication Key

Before proving the security of the entire protocol, we first propose a somewhat counter-intuitive lemma, which claims that the “view” of all parties up to the checking phase (i.e. before executing any PartialVerify) cannot break the shuffle authentication key β . This seems counter-intuitive, since in a typical MPC, the joint view of all parties reveals all secrets. The point here is that this “view” does not contain the values of β and $\mathbf{r}'_i$, which is hidden behind $\mathcal{F}_{\text{MPC}}$.

Lemma 4. *Consider the following view:*

$$\text{view} = \left\{ \begin{array}{c} \mathbf{x} \\ \mathbf{z}_1 \ \mathbf{z}_2 \ \cdots \ \mathbf{z}_n \\ \pi_1 \ \pi_2 \ \cdots \ \pi_n \end{array} \right\}.$$

This view consists of $\mathbf{x}$, all secret permutations held by parties and all opened values during both the offline and online phases of the protocol, before entering any PartialVerify. Note that all $\mathbf{y}_i$ can be deduced from the view.

This lemma claims that the view is independent of β , i.e.

$$I(\beta; \text{view}) = 0.$$

Or equivalently,

$$\Pr[\beta = \beta' \mid \text{view}] = \frac{1}{|\mathbb{F}|},$$

for any $\beta' \in \mathbb{F}$.

Proof. This view can be expanded as

$$\text{view} = \begin{pmatrix} \mathbf{x} \\ \mathbf{x} - \mathbf{r}_1 & \pi_1(\mathbf{r}_1) - \mathbf{r}_2 & \cdots \\ \beta\mathbf{x} - \beta\mathbf{r}_1 - \mathbf{r}'_1 & \pi_1(\beta\mathbf{r}_1 + \mathbf{r}'_1) - \beta\mathbf{r}_2 - \mathbf{r}'_2 & \cdots \\ \pi_1 & \pi_2 & \cdots \end{pmatrix},$$

where the second row is $\mathbf{z}_i^1$ and third row $\mathbf{z}_i^2$. Note that we may further simplify this view and replace it with

$$\text{view} = \begin{pmatrix} \mathbf{x} \\ \mathbf{r}_1 & \mathbf{r}_2 & \cdots \\ \beta\mathbf{x} - \beta\mathbf{r}_1 - \mathbf{r}'_1 & \pi_1(\beta\mathbf{r}_1 + \mathbf{r}'_1) - \beta\mathbf{r}_2 - \mathbf{r}'_2 & \cdots \\ \pi_1 & \pi_2 & \cdots \end{pmatrix},$$

since this view can deduce the second row in the former view, and vice versa.

Now it should be clear that this view is independent of β , since the third row is the only row containing term β , yet each term is masked by an independent random vector $\mathbf{r}'_i$. Stated otherwise, for each possible assignment of β , there is exactly one set of assignment to all $\mathbf{r}'_i$ such that the above view does not change.

Hence, the view is independent of β . $\square$

What does this lemma imply? Not surprisingly, it means that however the corrupted parties act, they cannot deduce any information about β , prior to entering the checking phase. This is formally stated in the following theorem.

Theorem 3. *Before execution of any PartialVerify protocol, the adversary cannot deduce any information about β from the combined protocol Shuffle, however it acts. Hence, the probability of the adversary correctly guessing β is $1/|\mathbb{F}|$, the same as randomly drawing a field element.*

Proof. Consider the adversary's view. Firstly in the offline phase $\text{Shuffle}_{\text{off}}$, the view is exactly $\{\mathbf{z}_i\}_{i \in T \setminus \{1\}}$ and $\{\pi_i\}_{i \in T}$. This is due to Lemma 1. As the offline phase consists only of calls to $\mathcal{F}_{\text{MPC}}$ subroutines, however the adversary acts, it cannot learn anything beyond this view.

In the online phase, the adversary $\mathcal{A}$ learns the message of honest party P_k , which is $\mathbf{y}_k$. Recall that

$$\mathbf{y}_k = \pi_k(\mathbf{y}_{k-1} + \mathbf{z}_k),$$

by the design of the protocol. Whatever $\mathbf{y}_{k-1}$ the adversary may choose, the information available in $\mathbf{y}_k$ is no more than π_k and $\mathbf{z}_k$. Note that in the online phase, before entering the checking phase, the only chance for the adversary to learn any information is to choose possibly arbitrary $\mathbf{y}_k$ and observe the output of honest party P_k .

Recall that in Lemma 4, the adversary is assumed to learn precisely $\mathbf{x}$ and all the π_i and $\mathbf{z}_i$ for $i \in [n]$. The lemma has stated that, this is still insufficient to deduce any information about β due to the lack of knowledge of $\mathbf{r}'_i$. $\square$

Remark. In the theorem above we state that even if the adversary learns both π_i and $\mathbf{z}_i$ of honest P_i , it cannot learn anything about β before entering the checking phase. The fact that the adversary (in fact) cannot learn anything about π_i of honest P_i is later shown by constructing a UC simulator that is ignorant of π_i .

C.3 Enforcement of Honest Behaviors in the Online Phase

The enforcement of honest behaviors of the adversary means that the adversary cannot send any wrong messages (different from what is specified by protocol) to any honest party, otherwise the protocol aborts with overwhelming probability in the first check related to that message. We begin by proving an auxiliary result regarding the security of the correlation test protocol.

Theorem 4 (Knowledge-of- β Barrier). *In the sequential invocations of PartialVerify, if the protocol has not aborted for P_{i+1} , then with probability at least $1 - \frac{1}{2^k}$, P_{i+1} has broadcast correct w_1 and w_2 (that are identical to what would be broadcast if all parties are honest), and the MAC key β remains hidden.*

Proof. Let's prove by induction. The basic case is the invocation of verification protocol for P_2 .

Without loss of generality, consider a malicious P_2 . By Theorem 3, prior to the verification phase, the adversary does not have any knowledge about β . By examining the protocol, it is clear that the adversary remains ignorant of β upon broadcasting w_1 and w_2 , since c_2 opened to it is independent of β . Also note that, if P_2 is malicious, it must know correct w_1 and w_2 , regardless of whether P_1 is honest or not. However, for one who knows (w_1, w_2) and is ignorant of β , forging valid $(\hat{w}_1, \hat{w}_2)$ different from (w_1, w_2) is equivalent to guessing β , since

$$\beta = \frac{\hat{w}_2 - w_2}{\hat{w}_1 - w_1}.$$

Therefore, if the protocol exits normally, then with probability at least $1 - \frac{1}{2^k}$ that correct w_1 and w_2 are broadcast.

By a similar argument and induction, as long as the protocol does not abort for P_i , the adversary remains ignorant of β . And due to its ignorance, the w_1 and w_2 broadcast so far must all be correct, except with negligible probability. $\square$

The next theorem enforces the correctness of all $\mathbf{y}_i$, hence an honest online phase.

Theorem 5. *If the adversary $\mathcal{A}$ sends any wrong $\mathbf{y}_i$ that is different from what is specified by the protocol to honest P_{i+1} , then the protocol will abort with probability at least $1 - \frac{m}{2^\kappa}$.*

Further, if $\mathbf{y}_n$ broadcast by P_n is different from what is specified by the protocol, the protocol aborts with probability at least $1 - \frac{1}{2^\kappa}$.

Hence, if the tests pass, it follows with overwhelming probability that all $\mathbf{y}_i$ sent to honest parties P_{i+1} are in accordance with the protocol, as well as the $\mathbf{y}_n$ broadcast by P_n .

Proof. In the online phase $\text{Shuffle}_{\text{on}}$, it is required that the test

$$\text{PartialVerify}(P_{i+1}, \mathbf{y}_i^1, \mathbf{y}_i^2, \langle \beta \rangle, \langle c_{i+1} \rangle, \langle d_{i+1} \rangle)$$

passes, for all honest P_{i+1} . This can be divided into two cases:

1. $\beta \mathbf{y}_i^1 = \mathbf{y}_i^2 + \pi_i(\mathbf{r}'_i)$, i.e. the message satisfies correct correlation, and the protocol exits normally.
2. $\beta \mathbf{y}_i^1 \neq \mathbf{y}_i^2 + \pi_i(\mathbf{r}'_i)$, but the protocol does not abort.

By Lemma 3, the second case happens with probability at most $(m-1)/2^\kappa$. Also, it is required that

$$\text{Verify}(\mathbf{y}_n^1, \mathbf{y}_n^2, \langle \beta \rangle, \langle \pi_n(\mathbf{r}'_n) \rangle)$$

passes, which by Lemma 2 means precisely $\beta \mathbf{y}_n^1 = \mathbf{y}_n^2 + \pi_n(\mathbf{r}'_n)$. Hence, it remains to prove that the adversary cannot forge wrong message with correct correlation, except with probability at most $1/2^\kappa$.

Assume for contradiction that the adversary somehow forges a different $\hat{\mathbf{y}}_i \neq \mathbf{y}_i$ for some $i \in S \subset [n]$, such that P_{i+1} is honest (if P_{i+1} exists) and

$$\beta \hat{\mathbf{y}}_i^1 = \hat{\mathbf{y}}_i^2 + \pi_i(\mathbf{r}'_i).$$

Let's fix i as the least element in S , so P_{i+1} is the first "victim". Since there are no wrong messages sent to honest party before i , (malicious) P_i must know correct $\mathbf{y}_i$ to send, which satisfies

$$\beta \mathbf{y}_i^1 = \mathbf{y}_i^2 + \pi_i(\mathbf{r}'_i).$$

By subtracting the two equations, the adversary obtains

$$\beta(\mathbf{y}_i^1 - \hat{\mathbf{y}}_i^1) = \mathbf{y}_i^2 - \hat{\mathbf{y}}_i^2.$$

Since $\mathbf{y}_i \neq \hat{\mathbf{y}}_i$, it has to be the case that

$$\mathbf{y}_i^1 \neq \hat{\mathbf{y}}_i^1.$$

Hence, the adversary will be able to find some index $j \in [m]$, such that

$$\beta(\mathbf{y}_i^1(j) - \hat{\mathbf{y}}_i^1(j)) = \mathbf{y}_i^2(j) - \hat{\mathbf{y}}_i^2(j) \text{ s.t. } \mathbf{y}_i^1(j) - \hat{\mathbf{y}}_i^1(j) \neq 0,$$

where $\mathbf{y}_i^1(j)$ denotes the j -th entry of the vector. This allows the adversary to solve β explicitly as

$$\beta = \frac{\mathbf{y}_i^2(j) - \hat{\mathbf{y}}_i^2(j)}{\mathbf{y}_i^1(j) - \hat{\mathbf{y}}_i^1(j)}.$$

Observe that the action of sending $\hat{\mathbf{y}}_i$ happens before the verification phase. By Theorem 3, upon sending $\hat{\mathbf{y}}_i$, the adversary could not learn β from its view, yet here it would have already solved it explicitly. Thus, the assumption that “the adversary could somehow forge some different $\hat{\mathbf{y}}_i$ that satisfies the correlation” will not happen with probability better than the same probability of guessing β correctly, which is at most $1/2^\kappa$.

Thus, if the protocol does not abort after all the checks, with overwhelming probability $p \geq 1 - \frac{m}{2^\kappa}$ all $\mathbf{y}_i$ sent to honest parties are correct, in the sense that it is precisely the one produced by honestly following the protocol. $\square$

Hence, we conclude in the following corollary that the honest behavior of the adversary is enforced in both the offline and online phases.

Corollary 1. *The shuffle protocol enforces honest behaviors in both the offline phase and the online phase, in the sense that any misbehavior deviating from the protocol will cause abort with overwhelming probability.*

In particular, in the online phase, this means that sending any incorrect messages (e.g. $\mathbf{y}_i$, w_1 , w_2) to honest parties will lead to immediate abort in the first subsequent check related to that message, with overwhelming probability.

Proof. This is obtained by combining Theorems 4 and 5. $\square$

C.4 UC Simulator

We are now able to prove the UC security of the shuffle protocol. First, we formally define the two games in the real and ideal worlds, which form the basis for defining the security of the protocol.

Definition 8 (Real and Ideal Worlds). *In the real execution of the shuffle protocol, at the start of the game, the environment $\mathcal{E}$ sends $\mathbf{x}$ to $\mathcal{F}_{\text{MPC}}$, which stores it locally as $\langle \mathbf{x} \rangle$. $\mathcal{E}$ also sends π_i to honest P_i , as P_i ’s input for protocol Shuffle. Then the adversary $\mathcal{A}$ corrupting $T \subseteq [n]$ begins executing the protocol Shuffle with honest parties, while receiving instructions from and sending information to $\mathcal{E}$. At the end of the protocol, if the parties do not abort, $\mathcal{F}_{\text{MPC}}$ sends its output of Shuffle to $\mathcal{E}$, which is (by design) $\pi(\mathbf{x})$. We denote this process by $\mathcal{E} \hookrightarrow \Pi_{T, \mathcal{A}}$.*

In the ideal world, $\mathcal{A}$ is replaced by simulator $\mathcal{S}$, which plays the role of all corrupted parties. $\mathcal{F}_{\text{MPC}}$ is equipped with a new command Π_{shuffle} , which requires parties to agree on $\langle \mathbf{x} \rangle$ to be shuffled, and takes as input π_i from P_i . The modified $\mathcal{F}_{\text{MPC}}$ is hence denoted as $\mathcal{F}_{\text{shuffle}}$. At the start of the game, $\mathcal{E}$ sends $\mathbf{x}$ to the ideal functionality $\mathcal{F}_{\text{shuffle}}$. $\mathcal{E}$ also sends π_i for honest party P_i to $\mathcal{F}_{\text{shuffle}}$, as part of the inputs for Π_{shuffle} . Then $\mathcal{E}$ interacts with $\mathcal{S}$ as if with $\mathcal{A}$, controlling what are

sent from corrupted parties and receiving what are received by corrupted parties. If the game does not abort, $\mathcal{E}$ receives from $\mathcal{F}_{\text{shuffle}}$ the output vector. Denote this process by $\mathcal{E} \sqsubseteq \mathcal{F}_{\bar{T}, \mathcal{S}}$, where $\mathcal{F}$ refers to $\mathcal{F}_{\text{shuffle}}$.

At the end of each game, $\mathcal{E}$ outputs a single bit, representing its judgement of whether the execution corresponds to the real-world game.

The protocol `Shuffle` UC-securely implements Π_{shuffle} in $\mathcal{F}_{\text{MPC}}$ -hybrid model, if for every $\mathcal{E}$ and adversary $\mathcal{A}$, there exists a probabilistic polynomial-time simulator $\mathcal{S}$, such that

$$|\Pr [1 \leftarrow (\mathcal{E} \sqsubseteq \Pi_{\bar{T}, \mathcal{A}})] - \Pr [1 \leftarrow (\mathcal{E} \sqsubseteq \mathcal{F}_{\bar{T}, \mathcal{S}})]| < \epsilon = O(\frac{m}{2^\kappa}),$$

for a statistical secure parameter κ fixed a priori and length m of $\mathbf{x}$.

Remark. For the protocol to achieve UC security, it is necessary to let $\mathcal{E}$ choose the $\mathbf{x}$ to be shuffled, and then obtain the shuffled result. Since in practice, it could be the case that $\mathbf{x}$ comes directly from a preceding Π_{input} command and will be opened directly after. The security relies on the fact that the simulator $\mathcal{S}$, while being completely ignorant of $\mathbf{x}$ and the π_i chosen by the honest parties, can still create the view of the adversary $\mathcal{A}$ in real world that is indistinguishable to any $\mathcal{E}$. When $\mathcal{F}_{\text{shuffle}}$ is replaced by a real-world implementation of the multiparty computation, the universal composition theorem will guarantee the overall security.

Theorem 6. Assume the protocols are working in a field of size no smaller than 2^κ , where κ is a statistical security parameter chosen arbitrarily.

Then the protocol `Shuffle` is universally composable secure in $\mathcal{F}_{\text{MPC}}$ -hybrid model, meaning that for any environment $\mathcal{E}$ and adversary $\mathcal{A}$, there exists a probabilistic polynomial-time simulator $\mathcal{S}$ such that

$$|\Pr [1 \leftarrow (\mathcal{E} \sqsubseteq \Pi_{\bar{T}, \mathcal{A}})] - \Pr [1 \leftarrow (\mathcal{E} \sqsubseteq \mathcal{F}_{\bar{T}, \mathcal{S}})]| < \epsilon = O(\frac{m}{2^\kappa}).$$

Proof. We construct a simulator for a dummy adversary $\mathcal{A}$, which forwards all messages to $\mathcal{E}$ and delegates all decisions to it. At a high level, the simulator only needs to sample the offline correlation independently of $\mathbf{x}$, leveraging the indistinguishability of $\mathbf{z}_i$, and deduce all corrupted party inputs for the ideal functionality. Below, we detail this simulation.

At the beginning of the protocol, $\mathcal{E}$ decides the input $\mathbf{x}$ and the input π'_i for honest party P_i , and sends all of them to the ideal functionality $\mathcal{F}_{\text{shuffle}}$.

The simulator $\mathcal{S}$ works as follows. In the offline phase of the shuffle, $\mathcal{S}$ first draws β uniformly from $\mathbb{F}$. It interacts with $\mathcal{E}$ and obtains the purported input π'_i for corrupted party P_i . The simulator can obtain π'_i , because it is the input of corrupted party P_i for Π_{perm} , which means that $\mathcal{E}$ needs to send π'_i explicitly to $\mathcal{S}$ upon calling Π_{perm} . Then $\mathcal{S}$ simulates the output of the offline phase with β and π_i , acts as if each honest P_i has chosen identical permutation. It draws uniformly random vectors $\mathbf{r}_i$, $\mathbf{r}'_i$ and c_i , and computes $\mathbf{z}_i$ and d_i as is specified

by the protocol. To be specific, $\mathcal{S}$ hence generates and stores locally the entire shuffle correlation (in plaintext)

$$\text{cor} = \begin{pmatrix} \langle \beta \rangle & \langle \mathbf{r}_1 \rangle & \langle \beta \mathbf{r}_1 \rangle & \langle \mathbf{r}'_1 \rangle & \langle \pi_n(\mathbf{r}_n) \rangle \\ \pi_1 & \pi_2 & \pi_3 & \cdots & \pi_n \\ \langle \pi_1(\mathbf{r}'_1) \rangle & \langle \pi_2(\mathbf{r}'_2) \rangle & \langle \pi_3(\mathbf{r}'_3) \rangle & \cdots & \langle \pi_n(\mathbf{r}'_n) \rangle \\ & \langle c_2 \rangle & \langle c_3 \rangle & \cdots & \langle c_n \rangle \\ & \langle d_2 \rangle & \langle d_3 \rangle & \cdots & \langle d_n \rangle \\ & \mathbf{z}_2 & \mathbf{z}_3 & \cdots & \mathbf{z}_n \end{pmatrix}$$

where

$$\pi_i = \begin{cases} \text{identical permutation} & \text{if } P_i \text{ is honest,} \\ \pi'_i & \text{otherwise.} \end{cases}$$

When a corrupted party P_i is to receive $\mathbf{z}_i$, $\mathcal{S}$ simply sends the $\mathbf{z}_i$ it computes internally to $\mathcal{E}$. $\mathcal{S}$ also sends all π'_i for corrupted parties to $\mathcal{F}_{\text{shuffle}}$. The simulation up to this point is perfect, since $\mathbf{z}_i$ is uniformly random in both real and ideal world games.

Then in the online phase, $\mathcal{S}$ simulates the message of each P_i . If P_1 is honest, $\mathcal{S}$ sends, on behalf of P_1 ,

$$\mathbf{y}_1 := \pi_1(\mathbf{z}_1) = \mathbf{z}_1 = (\mathbf{r}_1, -\beta \mathbf{r}_1 - \mathbf{r}'_1),$$

treating $\mathbf{x}$ as a zero vector. If P_1 is dishonest, $\mathcal{S}$ informs $\mathcal{E}$ that P_1 has received the above $\mathbf{z}_1$, delegating the decision of $\mathbf{y}_1$ to $\mathcal{E}$. Then for each P_i , if P_i is honest, since $\mathcal{S}$ knows the entire shuffle correlation, $\mathcal{S}$ acts on behalf of P_i and sends honestly

$$\mathbf{y}_i := \pi_i(\mathbf{y}_{i-1} + \mathbf{z}_i) = \mathbf{y}_{i-1} + \mathbf{z}_i$$

to P_{i+1} . For dishonest P_i , $\mathcal{S}$ leaves $\mathcal{E}$ to decide the message to send.

After P_n broadcasts $\mathbf{y}_n$, for each $i \geq 2$, $\mathcal{S}$ simulates PartialVerify. $\mathcal{S}$ first opens c_i to P_i . If P_i is honest, $\mathcal{S}$ simulates it honestly by broadcasting

$$w_b = \sum_{j=1}^m c_{i+1}^{j-1} \mathbf{y}_i^b(j).$$

If P_i is corrupted, $\mathcal{E}$ decides w_b to be broadcast. $\mathcal{S}$ then opens the resulting d , and informs $\mathcal{E}$ of an abort if $d \neq 0$.

If the above process does not result in an abort, $\mathcal{S}$ continues to simulate Verify. If the protocol ends without abort, and $\mathcal{E}$ does not request $\mathcal{S}$ to abort, $\mathcal{S}$ instructs $\mathcal{F}_{\text{shuffle}}$ to proceed. $\mathcal{F}_{\text{shuffle}}$ then proceeds to send

$$\pi(\mathbf{x}) := \pi'_n \circ \pi'_{n-1} \circ \cdots \circ \pi'_1(\mathbf{x})$$

to $\mathcal{E}$, where recall that π'_i of honest P_i is sent earlier from $\mathcal{E}$ to $\mathcal{F}_{\text{shuffle}}$, and π'_i of corrupted P_i is sent earlier from $\mathcal{S}$.

To show that this simulator produces a view statistically indistinguishable from the view of the adversary in the real world game, note that by Corollary

1, if $\mathcal{E}$ misbehaves (by sending wrong messages to any honest party), the protocol aborts with overwhelming probability $p \geq 1 - \frac{m}{2^k}$ in the next (related) check in both real and ideal world. So if $\mathcal{E}$ misbehaves, the view will be statistically indistinguishable, since if the protocol aborts, all values opened so far are uniformly random field elements, and in both worlds, the protocol aborts with overwhelming probability.

If $\mathcal{E}$ acts honestly and the protocol finishes without abort, $\mathcal{E}$ will receive permuted $\pi(\mathbf{x})$. Tracing back, due to its lack of knowledge of $\mathbf{r}_i$ and $\mathbf{r}'_i$, all values in its view appear consistent with an honest execution. To be specific, for each possible assignment to β and $\{\pi_i\}_{i \in T}$, $\mathcal{E}$ can find exactly one set of assignment to all $\mathbf{r}_i$ and $\mathbf{r}'_i$, so that all $\mathbf{z}_i$ and $\mathbf{y}_i$ are consistent with its view and the result is indeed $\pi(\mathbf{x})$. This is the same as the semi-honest case, in proof of Theorem 2. $\square$

D Existing Shuffle Correlation

We note that before our work, the concept of shuffle correlation has already been proposed by Eskandarian and Boneh [16] and Chen et al. [9]. As their constructions are similar in many aspects, e.g. both are based on the share translation protocol of Chase et al. [7], both are specialized for additive secret sharing, both are semi-honest only, etc., below we focus on reviewing the construction of Eskandarian and Boneh.

As Song et al. [35] point out, the construction of [16] is vulnerable to selective abort attack. In this section, we briefly review the construction of [16] and the attack of [35], and compare them with our construction.

D.1 Semi-honest Shuffle Correlation of [16]

The basic idea of [16] is similar to our idea discussed in Section 4. In the online phase, after opening $\mathbf{x} + \mathbf{r}$ to P_1 , each party P_i receives a masked vector $\mathbf{y}_{i-1}$ from P_{i-1} , applies its own permutation π_i on it, adds a random mask $\mathbf{r}_i$, and then sends to P_{i+1}

$$\mathbf{y}_i \leftarrow \pi_i(\mathbf{y}_{i-1}) + \mathbf{r}_i.$$

Following this idea, the main goal of the offline phase is to generate

$$\llbracket \Delta \rrbracket = \pi_n(\pi_{n-1}(\cdots \pi_1(\llbracket \mathbf{r} \rrbracket) + \llbracket \mathbf{r}_1 \rrbracket \cdots) + \llbracket \mathbf{r}_{n-1} \rrbracket) + \llbracket \mathbf{r}_n \rrbracket.$$

It is shown by Eskandarian and Boneh [16] that the above shuffle correlation can be generated in $O(1)$ rounds, with the help of a two-party share translation protocol [7]. The share translation protocol is a two-party protocol that outputs $(\pi_i, \Delta_{i,j})$ to party P_i and $(\mathbf{a}_{i,j}, \mathbf{b}_{i,j})$ to party P_j , such that π_i is the input of P_i and $\Delta_{i,j} = \pi_i(\mathbf{a}_{i,j}) + \mathbf{b}_{i,j}$. Given such a primitive, [16] defines their shuffle correlation as follows.

Definition 9 (Shuffle Correlation, from [16]). *The n -party shuffle correlation is a situation where for $i = 1, \dots, n$, P_i holds:*

1. random vectors $\mathbf{a}_i, \mathbf{b}_i, \mathbf{a}'_i \in \mathbb{F}^m$ and
2. a random permutation $\pi_i : [m] \rightarrow [m]$.

P_n holds in addition a vector $\Delta_n \in \mathbb{F}^m$, such that

$$\Delta_n = \pi_n(\dots(\pi_2(\pi_1(\sum_{i=2}^n \mathbf{a}_i) + \mathbf{a}'_1) + \mathbf{a}'_2) \dots + \mathbf{a}'_{n-1}) - \sum_{i=1}^{n-1} \mathbf{b}_i.$$

Note that this Δ_n has a form very close to the one we mentioned above, and indeed, they are equivalent up to some simple reformations. To generate Δ_n in $O(1)$ rounds, [16] further specifies

1. $\mathbf{a}_i \leftarrow \mathbf{a}_{1,i}, \mathbf{b}_i \leftarrow \mathbf{b}_{n,i}$.
2. $\mathbf{a}'_i \leftarrow \sum_{j \in [n] \setminus \{i+1\}} \mathbf{a}_{i+1,j} - \sum_{j \in [k] \setminus \{i\}} (\mathbf{b}_{i,j} + \Delta_{i,j})$.
3. $\Delta_n \leftarrow \sum_{j \in [n-1]} \Delta_{n,j}$.

Wherein each $(\mathbf{a}_{i,j}, \mathbf{b}_{i,j}, \Delta_{i,j})$ is generated by invoking the share translation protocol of [7] for P_i and P_j . To compute $\mathbf{a}'_i$, each party P_i shares each $\llbracket \mathbf{a}_{j,i} \rrbracket$, $\llbracket \mathbf{b}_{j,i} \rrbracket$, $\llbracket \Delta_{i,j} \rrbracket$, which requires in addition $O(n^3 m)$ communication for sharing $O(n^2)$ random values.

As seen above, it is clear that the definition of shuffle correlation in [16] is specialized for additive secret sharing, and that its generation relies on the two-party share translation protocol of [7]. Their construction requires an expensive translation from two-party randomness to n -party randomness, which results in additional $O(n^3 m)$ communication and computation overhead, making the overall complexity $O(n^2 m \log m + n^3 m)$. This complexity is suboptimal, as executing the share translation protocol of [7] for n^2 times requires only $O(n^2 m \log m)$ complexities.

In contrast to this approach, our definition (c.f. Definition 3) for semi-honest shuffle correlation is designed for and compatible with a general MPC framework. Specifically, we define⁷

$$\mathbf{s} = \pi_n(\dots \pi_3(\pi_2(\pi_1(\mathbf{r}) - \mathbf{z}_2) - \mathbf{z}_3) \dots - \mathbf{z}_n),$$

which we then implement as

$$\begin{aligned} \llbracket \mathbf{r}_i \rrbracket &\leftarrow \Pi_{\text{rand}}(), \\ \llbracket \mathbf{z}_i \rrbracket &\leftarrow \pi_{i-1}(\llbracket \mathbf{r}_{i-1} \rrbracket) - \llbracket \mathbf{r}_i \rrbracket, \text{ for } i = 2, 3, \dots, n, \\ \llbracket \mathbf{s} \rrbracket &\leftarrow \llbracket \pi_n(\dots \pi_3(\pi_2(\pi_1(\mathbf{r}_1) - \mathbf{z}_2) - \mathbf{z}_3) \dots - \mathbf{z}_n) \rrbracket = \llbracket \pi_n(\mathbf{r}_n) \rrbracket. \end{aligned}$$

Clearly, this definition and implementation is based on n -party randomness, which avoids the usage of two-party randomness and thus avoids the $O(n^3 m)$ complexity term. Furthermore, it also offers a clear approach for generating the correlation. As now $\mathbf{s} = \pi_n(\mathbf{r}_n)$, we can easily parallelize the invocations to Π_{perm} and accelerate the correlation generation.

⁷ Note that the definition of $\mathbf{s}$ is slightly different from the Δ above. The two definitions are equivalent up to some reformations.

Thus, besides its simplicity and compatibility, our definition also achieves a better offline communication and computation complexity compared to [16] in semi-honest additive secret sharing setting — the secret sharing scheme for which [16] has been specialized.

D.2 Malicious Security and Selective Abort Attack

To achieve malicious security, [16] suggests that parties generate and share in addition a secret random vector $\mathbf{g}$ and compute and share (via MPC multiplication) $\mathbf{h}$, such that $\mathbf{h}(i) = \mathbf{x}(i) \cdot \mathbf{g}(i)$ (i.e. Hadamard product). By shuffling $\mathbf{g}, \mathbf{h}$ along with $\mathbf{x}$, and checking in the end if $\mathbf{h}'(i) = \mathbf{x}'(i) \cdot \mathbf{g}'(i)$ for all entry i , it is suggested that the protocol is secure against malicious adversary.

However, as Song et al. [35] point out, this construction is vulnerable to selective abort attack. For example, suppose P_1 and P_3 are corrupted. The adversary may choose random indices i and j , and let P_1 send $\mathbf{y}_1 + \boldsymbol{\delta}_i$ instead of $\mathbf{y}_1$, where $\boldsymbol{\delta}_i$ is a one-hot vector with its i -th entry being 1. After receiving $\mathbf{y}_2$ from P_2 , P_3 first adjusts it by computing $\mathbf{y}_2 - \boldsymbol{\delta}_j$, and acts honestly hereafter. If P_2 happens to choose a permutation π_2 with $\pi_2(j) = i$ (with non-negligible probability $1/m$), the behavior of P_3 corrects the error introduced by P_1 . Hence, if parties do not abort in the final check, the adversary learns $\pi_2(j) = i$ without being detected.

At first glance, it seems quite easy to defend against such an attack: simply check if the Hadamard product relation remains inside each message $\mathbf{y}_i$ sent by P_i , and everything seems to work out. However, such a check is costly. To protect the underlying data, the message P_{i+1} receives from P_i is masked with uniform random mask, which prevents P_{i+1} from locally checking its correctness. If P_{i+1} wants to check if the message is correct, i.e. if the Hadamard product relation is maintained, P_{i+1} needs to share the entire message back to all parties, and then all parties, after jointly removing the random masks, carry out $O(m)$ MPC multiplications over all $O(m)$ entries. — This is already $O(n^2m)$ online communication and computation, as an MPC multiplication operation typically costs $O(n)$ overheads, which is repeated for n parties over m entries.

Fortunately, we observe that this overhead can be avoided by carefully designing the correlation embedded in the messages. Suppose the message sent from P_i to P_{i+1} is $\mathbf{y}_i$. We now require P_i to send in addition a message $\hat{\mathbf{y}}_i$ that acts as a MAC, which satisfies

$$\beta \mathbf{y}_i = \hat{\mathbf{y}}_i + \mathbf{s}_i,$$

with $\langle \beta \rangle$ and $\langle \mathbf{s}_i \rangle$ stored at $\mathcal{F}_{\text{MPC}}$. Due to lacking β and vector $\mathbf{s}_i$, P_i will not be able to forge wrong message satisfying such relation. After P_n broadcasts $\mathbf{y}_n$ and $\hat{\mathbf{y}}_n$, all parties jointly check all previously sent $\mathbf{y}_i$ and $\hat{\mathbf{y}}_i$, and abort if any check fails.

The protocol up here still requires $O(n^2m)$ communication and computation, due to P_i broadcasting the $\mathbf{y}_{i-1}$ it receives, and all parties jointly computing $\langle \beta \rangle \mathbf{y}_i - \hat{\mathbf{y}}_i - \mathbf{s}_i$.

Moreover, these checks can be batched, in that P_{i+1} can sample a challenge λ , then turn to check if

$$\langle \beta \rangle \underbrace{\sum_{j=1}^m \lambda^{j-1} \mathbf{y}_i(j)}_{=u} = \underbrace{\sum_{j=1}^m \lambda^{j-1} \hat{\mathbf{y}}_i(j)}_{=v} + \underbrace{\sum_{j=1}^m \lambda^{j-1} \langle \mathbf{s}_i(j) \rangle}_{=w}.$$

Thus, P_{i+1} broadcasts only λ, u and v and all parties compute and open the subtraction of above two terms. This enables the check to be performed with only $O(n)$ communication, which is $O(n^2)$ for n parties. Furthermore, by moving the generation of λ (as now $\langle \lambda \rangle$) and w (as $\langle w \rangle$) to the offline phase, the online computation becomes also linear.

To see that this prevents the selective abort attack mentioned above, note that if malicious P_i sends incorrect $\mathbf{y}'_i$ and $\hat{\mathbf{y}}'_i$ to honest P_{i+1} , the protocol will abort during the checking phase. This is because without knowing β and $\mathbf{s}_i$, P_i cannot forge $\mathbf{y}'_i$ and $\hat{\mathbf{y}}'_i$ satisfying the above relation. Upon such abort, the input vector $\mathbf{x}$ is well protected, since the parties have been manipulating with $\mathbf{x} + \mathbf{r}$, which is independent of $\mathbf{x}$. Thus, our shuffle protocol either (1) aborts with $\mathbf{x}$ well hidden, or (2) completes with correct $\langle \pi(\mathbf{x}) \rangle$ shared, with secret π .

E Discussion

E.1 Security of MPC Functionalities

A practical concern is that in most MPC frameworks, the integrity checks of arithmetic operations—as well as the opening of values—may not be performed immediately. For example, in the original design of SPDZ [14], all opened values are required to be independent random values until the final output phase, which is preceded by a large batched check Π_{check} . This approach is appealing, as it amortizes the cost of integrity checks across the entire computation.

However, this concept mismatches the ideal functionality of $\mathcal{F}_{\text{MPC}}$ we have used throughout our paper. In particular, $\mathcal{F}_{\text{MPC}}$ is assumed to provide immediate integrity checks after each arithmetic operation and each opening of values. This assumption makes the security proof and analysis much cleaner, as we can always trust the correctness of all MPC operations performed up to any point of the protocol. As a result, this leaves a gap between the ideal functionality and real-world implementations of MPC frameworks.

There are two ways to fix this gap when implementing the protocol in real-world MPC frameworks. One is to execute the check for every Π_{open} in Algorithm 6, ensuring that all MPC opening operations up to this point are done correctly. While this aligns the real-world implementation with the ideal functionality, it also introduces significant overhead due to the frequent checks. As such a check typically costs $O(n^2)$ communication and computation [14, 17], this makes the online complexity $O(n^3 + nm)$.

The other approach is to execute the check only at the end of Algorithm 6, which is more efficient. The drawback is that π_i chosen by honest party P_i can

be leaked upon abort, as the adversary is now able to (temporarily) bypass the check of Algorithm 4, allowing the selective abort attack proposed by [35]. Note however, due to misbehaving inside the command of $\mathcal{F}_{\text{MPC}}$, this attack leads to abort upon Π_{check} with overwhelming probability. Hence, this in fact implements a (somewhat weaker) functionality of shuffle that only takes $\mathbf{x}$ as input. Under such definition, the leaked π_i corresponds only to a random internal state of the protocol, which is independent of the input $\mathbf{x}$ (that must be protected even upon abort). The online complexity hence remains $O(n^2 + nm)$.

We note that this second approach is also adopted by [35] in their maliciously secure shuffle protocol, which executes the SPDZ MAC check at the end of its shuffle protocol. In fact, this protocol is more conceptually a “shuffle protocol”: it either completes successfully with secret π and $\pi(\mathbf{x})$, or aborts and reveals nothing about $\mathbf{x}$. The permutation π , which is sampled independently uniformly during the protocol, is rather insignificant upon abort.

Note that in our security proof, we prove the security under the $\mathcal{F}_{\text{MPC}}$ -hybrid model, which guarantees the secrecy of π_i even upon abort. This corresponds to the first approach mentioned above, where each opening operation is equipped with an integrity check. The second approach can also be justified in a way similar to the original design of SPDZ [14], by noting that all opened values before the check are independent of the input $\mathbf{x}$. In this case, our security proof states that the adversary cannot learn information about π without being detected eventually.

E.2 Approximated Length of Vector

In the protocol described above, the offline phase $\text{Shuffle}_{\text{off}}$ is assumed to know m , the exact length of the input vector. In practice, however, it is often the case that m is not known in advance, as the input has not yet arrived. We now describe a simple extension that allows the protocol to operate with only an upper bound on m .

First, note that the shuffle protocol presented in this paper can trivially be extended to shuffle vectors of elements (instead of field elements) as long as the underlying permutation protocol Π_{perm} supports such an extension. This is achieved by replacing all occurrences of “vectors” with “matrices” in the protocol specification.

Therefore, the parties can first pad the input vector with dummy elements to extend its length to the upper bound m . Additionally, each element is extended to a vector of length 2, where the second entry is a public constant bit indicating whether the element is a dummy. The parties then execute the shuffle protocol on vectors of length 2. After the shuffle is completed, the second entries are opened, and dummy elements are discarded.

This padding technique can also be applied to shuffling an array of vectors, where only the upper bound of the length of each vector is known in advance.